

When Passing Tests Hides Vulnerabilities: An Empirical Study of Silent Failures in Agentic Systems*

Wenji Bai^{a,*}, Muhammad Waseem^a, Zeeshan Rasheed^a, Jaakko Peltonen^a and Pekka Abrahamsson^a

^aFaculty of Information Technology and Communication Sciences, Tampere University, Tampere, Finland

ARTICLE INFO

Keywords:

Agentic AI
Silent failures
LLM-based agents
Software security
Empirical study
Multi-Agent systems

ABSTRACT

LLM-based agents for automated code repair have received significant attention in recent years from both research and software engineering practice perspectives. However, limited attention has been paid to patches that pass syntactic and functional verification but still retain or introduce security vulnerabilities. The aim of this research is to systematically identify and categorize such silent failures in LLM-based agentic code repair.

We conducted an empirical study using 1,030 valid execution traces produced by seven agent frameworks with GPT-4o-mini across two security-focused datasets, SecurityEval and CVEfixes. Through three iterations of qualitative coding and manual verification, 170 confirmed silent failures were identified. The key results are: (i) Three main categories of silent failures were identified: Omission, Introduction, and Inadequacy. Omission accounts for 48.2% of the confirmed failures, Introduction for 30.6%, and Inadequacy for 21.2%. (ii) Ten fine-grained failure codes were classified under these three categories, showing how agents omit required security controls, apply incomplete defenses, or introduce new vulnerabilities during repair. (iii) Current test-passing evaluation and LLM-based reviewer roles were insufficient to expose or intercept these failures in the confirmed cases. (iv) Similar insecure solutions appeared across different frameworks, suggesting possible shared model-, prompt-, or task-level influences, while single-agent and multi-agent systems showed different failure profiles.

The results of this study will assist researchers and practitioners in improving the evaluation of LLM-based agentic code repair and developing targeted verification methods that go beyond functional correctness and cover all generated artifacts.

1. Introduction

Silent failures in automated software repair refer to patches that pass functional validation but still contain flaws, in particular security vulnerabilities. In automated program repair, a patch that compiles and passes all available test cases is conventionally termed *plausible*, but plausible patches are not necessarily correct [1]. We extend this concept into the security domain, and to systematically make the distinction explicit, we adopt a four-level verification hierarchy: syntactic validity (L0), functional validation (L1), static security analysis (L2), and exploitability assessment (L3). Thus, a patch may compile without errors (L0) and pass unit tests (L1), while still containing a vulnerability detectable through static security analysis (L2) or exploitability testing (L3) [2, 3]. Unlike overt errors such as compilation failures or test breakages, silent failures evade standard quality gates and can remain undetected and reach production systems.

This issue has become increasingly relevant with the rise of LLM-based agentic systems. Recent tools can handle multiple steps of the repair workflow, from fault diagnosis to patch implementation, marking a shift from earlier

code completion assistants [4–6]. Furthermore, multi-agent frameworks organize multiple LLMs into role-specialized pipelines that mirror human development teams [7–9]. These architectures use role specialization and interaction to improve output quality. However, recent studies report that multi-agent systems do not consistently outperform single-agent approaches, and their failure modes are not well characterized [10, 11]. These systems are increasingly deployed for security-sensitive tasks, including vulnerability patching and authentication repair [12], which increases the risk of undetected vulnerabilities.

Current evaluation practices can reinforce this risk. Benchmarks such as SWE-bench evaluate success primarily through test passing [13, 14]. In Continuous Integration and Continuous Delivery (CI/CD) pipelines, results are typically reported based solely on test passing criteria. When patches pass, reviewers may reasonably assume that agents have produced functionally correct repairs. However, an agent-generated patch can satisfy a narrowly scoped test case while retaining an exploitable vulnerability. For instance, an agent might resolve a data processing error to pass a functional test, yet continue to process untrusted user input without proper sanitization, a well-known vulnerability pattern [15, 16].

Prior work has explored related issues, but a systematic account of silent failures remains absent. Existing taxonomies categorize observable failure modes in multi-agent systems, yet they rely on explicit easily noticeable signals like system crashes or task non-completion; failures that produce no such obvious signals fall outside their scope [11].

*

*Corresponding author



wenji.bai@tuni.fi (W. Bai); muhammad.waseem@tuni.fi (M.

Waseem); zeeshan.rasheed@tuni.fi (Z. Rasheed); jaakko.peltonen@tuni.fi (J. Peltonen); pekka.abrahamsson@tuni.fi (P. Abrahamsson)

ORCID(S): 0000-0002-3283-3483 (W. Bai); 0000-0001-7488-2577 (M.

Waseem); 0000-0001-9655-3096 (Z. Rasheed); 0000-0003-3485-8585 (J.

Peltonen); 0000-0002-4360-2226 (P. Abrahamsson)

1

Similarly, current attribution frameworks offer mechanisms for identifying fault-inducing steps but still assume that failures are externally visible [17, 18]. In automated program repair, recent work has advocated for multi-dimensional evaluation incorporating security benchmarks and static analysis, yet these approaches have not been applied to characterize failures within multi-agent repair pipelines [19, 20].

Motivated by this gap, we formulate the following three research questions:

- *RQ1: What types of silent failures occur in LLM-based agent code repair, and how prevalent are they?*

Rationale: Existing taxonomies classify observable failure modes in multi-agent systems, but they primarily address explicit indications such as crashes, errors, or task non-completion. Silent failures, defined as patches that pass functional verification yet contain security deficiencies, remain insufficiently characterized. This question develops an empirically grounded taxonomy of silent failure types and quantifies their prevalence across frameworks and datasets, thereby establishing the basis for the subsequent vulnerability-context analyses.

- *RQ2: How do silent failures propagate through agent pipelines, and what role do architectural differences play?*

Rationale: Multi-agent frameworks employ specialized roles such as planners, engineers, and reviewers, but it remains unclear whether these role structures intercept security deficiencies or allow them to propagate across pipeline stages. This question traces silent failures through agent roles and pipeline stages, examines whether reviewer roles detect them, and compares propagation patterns between multi-agent and single-agent architectures. The goal is to clarify how architectural structure shapes the emergence, propagation, and interception of silent failures.

- *RQ3: Which vulnerability types and code contexts are most susceptible to silent failures, and what severity and exploitability levels do they exhibit?*

Rationale: Vulnerability characteristics may shape both the form silent failures take and their severity and exploitability, yet this relationship has received limited empirical attention. This question examines how silent failures distribute across CWE families, code locations, severity levels, and exploitability scores. The goal is to identify cross-dimensional patterns that can support targeted verification strategies rather than treating silent failures as a homogeneous defect class.

To address these research questions, we conduct an empirical study of silent failures across seven LLM-based agent frameworks and two security-focused datasets. Our analysis encompasses 1,030 execution traces, from which we identify 170 confirmed silent failures. Through iterative qualitative coding of execution traces and related artifacts, we develop

an empirically grounded taxonomy centered on *security-compliance decoupling* from mere unit-test passing. Specifically, this paper makes the following contributions:

- **An empirically grounded taxonomy of silent failures.** We propose an empirically grounded taxonomy that classifies silent failures along four dimensions (failure type, originator role, injection stage, and code location), extending beyond existing failure classifications by explicitly modeling failures that evade standard detection mechanisms.
- **A cross-framework empirical analysis.** We analyze 1,030 execution traces from seven agent frameworks across two security-focused datasets, providing quantitative evidence on the prevalence, distribution, and characteristics of silent failures.
- **Failure propagation analysis across agent roles.** By tracing silent failures through multi-agent execution logs, we identify how security omissions introduced at one pipeline stage propagate through subsequent stages, and compare propagation patterns between multi-agent and single-agent systems.
- **A public dataset.** We publicly release the study dataset online [21], including execution traces, generated patches, verification results, and taxonomy codes, to enable researchers and practitioners to access, replicate, and validate our findings.

The remainder of this paper is organized as follows. Section 2 describes our research methodology. Section 3 presents the empirical results. Section 4 discusses implications, and Section 5 addresses threats to validity. Section 6 reviews related work. Section 7 concludes with directions for future work.

2. Research Methodology

This section describes the research methodology used to identify, verify, and analyze silent failures in LLM-based agentic code repair. We first present the overall research design, followed by the dataset and agent framework selection criteria. We then describe the experimental setup, the multi-level verification framework, the manual review process, and the qualitative coding and reliability procedures used to construct and validate the taxonomy.

2.1. Research Design Overview

The study consists of two phases, as illustrated in Figure 1: (i) preparation of the shared experimental infrastructure, and (ii) iterative data collection and analysis. This design supports our three research questions on failure categorization, propagation mechanisms, and vulnerability susceptibility, while enabling consistent comparison across seven agent frameworks. The first phase covers dataset selection, framework selection, model configuration, the multi-level verification pipeline, and manual review criteria. The second

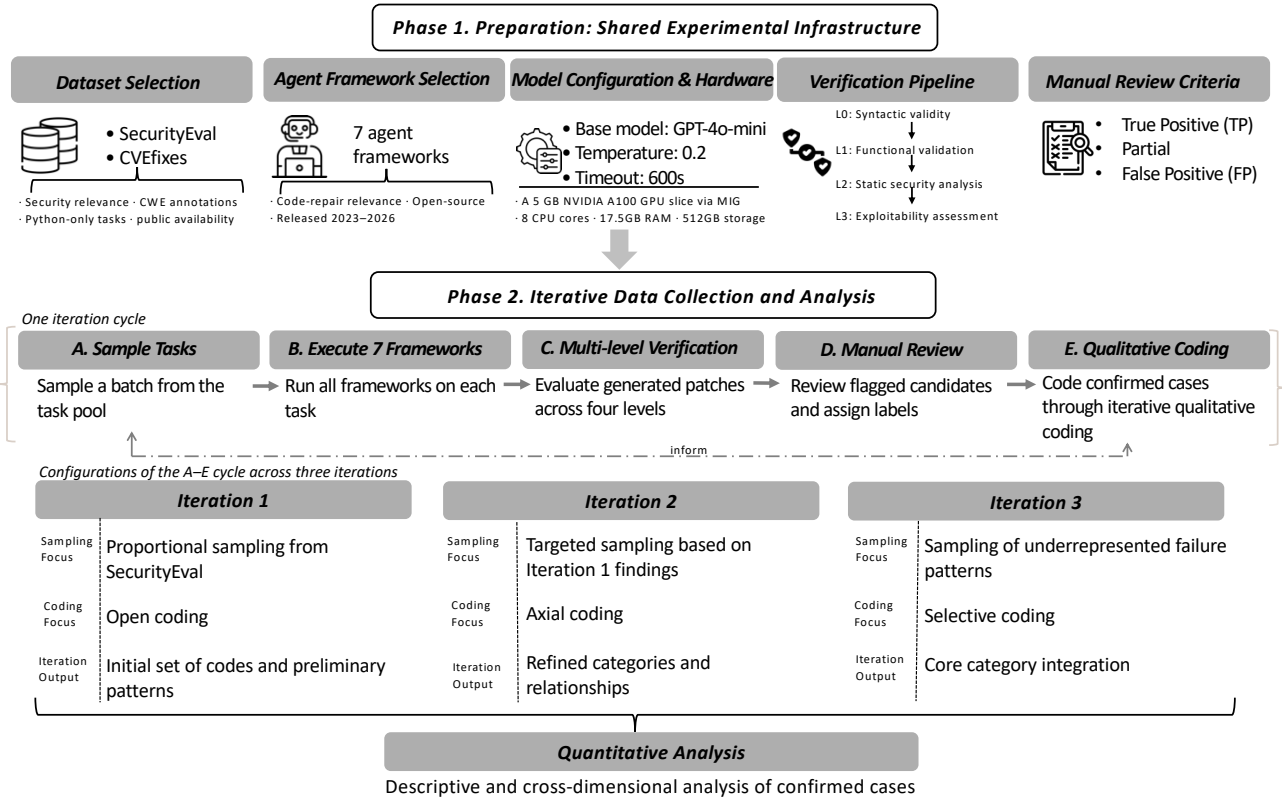


Figure 1: Overview of the research methodology.

Table 1

Summary of selected datasets. *Pool* denotes the eligible task count after language and format filtering; *Sampled* denotes the number of tasks drawn across three iterations.

	SecurityEval	CVEfixes	Total
Task Type	Code generation	Vulnerability repair	–
Original	130	5,365	5,495
Pool	121	1,115	1,236
Sampled	85	65	150
Labels	CWE	CVE/CWE	–

phase repeats the same procedure across three iterations: sampling tasks, executing all seven frameworks, evaluating generated patches through L0–L3 verification, manually reviewing flagged candidates, and qualitatively coding the confirmed cases. Observations from each iteration informed the sampling and analytical focus of the next.

2.2. Dataset Selection

To construct our dataset pool, we first defined four selection criteria: (i) security relevance, (ii) the availability of Common Weakness Enumeration (CWE) annotations to enable verification against expected vulnerability types, (iii) uniform execution conditions, requiring Python-only tasks so that all seven frameworks could be evaluated under identical conditions, and (iv) public availability. Based on these criteria, we selected two security-focused datasets, which together provided a combined pool of 1,236 Python

tasks. Table 1 summarizes the selected datasets, which are described below.

- **SecurityEval** [22] provides 130 predefined code generation prompts across 69 CWE categories. We retained the 121 Python tasks and excluded non-Python entries. Each task presents an insecure code context and requires the agent to generate a secure version, serving as a benchmark for initial pattern discovery and cross-framework comparison.
- **CVEfixes** [23] contains 5,365 real-world vulnerability fixes linking Common Vulnerabilities and Exposures (CVE) records to code changes in open-source projects. We filtered for Python single-file fixes with CWE annotations, yielding 1,115 tasks. CVEfixes tasks involve patching known vulnerabilities in existing production code and often require adding entirely new security logic rather than merely substituting an insecure API, representing real-world repair conditions.

2.3. Agent Framework Selection

To select the agent frameworks for this study, we first defined three criteria: (i) relevance to code repair, requiring that each framework be applicable to vulnerability repair tasks either by design or through reasonable adaptation; (ii) open-source availability; and (iii) recency, requiring that each framework has been released between 2023 and 2026 to

Table 2

Selected agent frameworks and their coordination architectures.

Framework	Type	Architecture	Selection Rationale
<i>Multi-Agent Systems</i>			
MetaGPT [8]	Multi	Assembly Line	Sequential role handoffs enable stage-level failure tracing
ChatDev [7]	Multi	Hierarchical Workflow	Phase-based workflow with explicit role transitions; validates pattern generalization across pipeline architectures
AutoGen [9]	Multi	Dynamic Conversation	Flexible orchestration with runtime role assignment; captures emergent failure patterns in less structured collaboration
AgentCoder [24]	Multi	Test-driven Loop	Independent test designer and coder roles; tests whether agent-generated tests mask vulnerabilities
<i>Single-Agent Systems</i>			
Mini SWE-Agent (MiniSWE) [4]	Single	ReAct Loop	Lightweight single-agent baseline; isolates coordination effects by removing inter-agent communication
Aider [25]	Single	Edit-feedback Loop	Structured diff output with lint/test feedback loops; represents developer-tool-style interaction
OpenHands [26]	Single	Edit-feedback Loop	CLI-based sandbox with file editing and shell execution in a single generalist loop

reflect recent advances in the field. Based on these criteria, we systematically selected seven agent frameworks representing distinct coordination patterns. Table 2 provides an overview of the selected frameworks and their coordination architectures.

2.4. Experimental Setup

This section outlines the model and prompt configuration, hardware platform, four-level verification framework, and manual review and confirmation criteria used in our study. In this setup, each task is independently processed by all frameworks to produce generated patches under a standardized configuration; every patch is then evaluated through a four-level verification pipeline (L0 to L3); finally, patches flagged at L2 or L3 are manually reviewed and confirmed as True Positive, Partial, or False Positive.

2.4.1. Model Configuration and Hardware

To reduce variability due to model differences, all frameworks used GPT-4o-mini (temperature 0.2) and were executed in isolated environments with a shared verification toolchain. We chose this model to balance response quality, cost, and scalability across the evaluation of seven frameworks. Experiments were run on a 5 GB NVIDIA A100 MIG slice with 8 CPU cores, 17.5 GB RAM, and 512 GB storage, with a 600-second timeout.

To maintain comparability across frameworks, we standardized the prompts by keeping the task content identical while adapting the format to each framework’s input schema. For example, ChatDev takes the task as a single project-level requirement string passed to its task parameter, while MiniSWE takes the same content as a user message inside a ReAct loop; in both cases, the core task description and code context remain identical, and only the input schema differs. At the dataset level, SecurityEval prompts included the original insecure code context, whereas CVEfixes prompts

paired the vulnerable code with a repair instruction specifying the target CVE/CWE. Beyond these dataset-specific differences, no additional security prompts were injected unless the task itself explicitly implied a security requirement.

2.4.2. Multi-Level Verification Framework

As outlined in Section 1, passing functional tests does not guarantee security correctness [1, 27]. We therefore evaluate every generated patch through a four-level verification pipeline:

- **Level 0 (L0), Syntactic validity.** The generated code parses without syntax errors.
- **Level 1 (L1), Functional validation.** The generated code passes available functional checks, including import resolution, structural correctness verification, and execution of associated test cases.
- **Level 2 (L2), Static security analysis.** The generated code is analyzed with Bandit (a tool to find Python code security issues, maintained by the Python Code Quality Authority). We run Bandit and map its findings to CWE identifiers using a predefined lookup table. A generated patch fails L2 if it contains any HIGH-severity finding, where severity is assigned based on CWE type.
- **Level 3 (L3), Exploitability assessment.** To support exploitability-oriented candidate screening, we apply a heuristic scoring mechanism that estimates risk based on the co-occurrence of user input sources and dangerous sinks, weighted by CWE severity [2]. Concretely, the score is computed from four signal groups extracted by pattern matching: (i) user-input sources, such as external inputs received from users, requests, or command-line arguments; (ii) dangerous

sinks, such as dynamic execution, shell invocation, unsafe deserialization, or security-sensitive database operations, each assigned a base weight reflecting its exploitation severity; (iii) indicators that the original vulnerability pattern targeted by the task is still present; and (iv) context-aware indicators capturing risky co-occurrences, such as a hard-coded credential near authentication logic. The total score increases when a user-input source and a dangerous sink co-occur in the same patch, and is further increased when the sink matches the target CWE of the task. The score is reduced when safe coding practices are present. In particular, bounded discounts are applied for safeguards such as input validation and sanitization, parameterized query construction, certificate or host verification, and secure replacement of known unsafe operations. The exploitability score R is computed as follows, clipped to the range $[0, 100]$:

$$R = \sum_{s \in \mathcal{I}} w_s + \sum_{j \in \mathcal{K}} b_j c t_j + 10 \delta_{\text{vuln}} - \min(2n_{\text{safe}}, 10), \quad (1)$$

where $\mathcal{I}$ and $\mathcal{K}$ are the input-source types and dangerous-sink matches detected in the patch, weighted by w_s and b_j respectively; $c = 1.5$ when an input source co-occurs with a sink and 1.0 otherwise; $t_j = 1.3$ when the sink CWE matches the task target CWE and 1.0 otherwise; $\delta_{\text{vuln}} = 1$ when the original target-CWE pattern is still present; and $n_{\text{safe}} \in \{0, \dots, 5\}$ is the number of safe-practice categories detected. The weights w_s follow three tiers by controllability (web request data 4, interactive and standard input 3, environment or file reads 2), and the weights b_j span 6 to 12 by exploitation severity, with dynamic code execution highest (12), command execution next (10), template rendering (9) and unsafe deserialization (8) in the middle, and formatted-string operations lowest (7 and 6). For patches spanning multiple files, R is taken as the maximum across files.

A generated patch whose total score reaches the threshold of 15 is treated as a silent failure candidate. We set the threshold to 15 and the associated signal weights in Eq. (1) based on calibration with Iteration 1 candidates. This configuration retained the high-risk source-sink co-occurrences observed during the initial calibration while reducing isolated low-severity patterns. Since all candidates undergo manual review before confirmation, the threshold serves as a sensitivity parameter for candidate selection rather than a final classification boundary.

We define patches that fail L0 or L1 as *explicit failures*, since their deficiencies are visible through standard quality gates. Patches that pass both levels but are flagged at L2 or L3 are treated as *silent failure candidates*: on the surface, they look like successful repairs because they are syntactically valid and pass available functional checks, yet later

security-oriented checks indicate that they may still contain or introduce a vulnerability. More formally, this is defined as:

$$SF_{\text{candidate}} = (L_0^{\text{pass}} \wedge L_1^{\text{pass}}) \wedge (L_2^{\text{fail}} \vee L_3^{\text{fail}}) \quad (2)$$

In other words, a patch is considered a silent failure candidate if it appears valid and passes available functional checks, but still raises concerns in later security checks.

As an illustrative case, consider a SecurityEval command-injection task that required repairing code that built and executed a shell command from external input. The agent-generated patch added a length check and renamed surrounding variables, but retained the original shell-execution call. The patch passed L0 and L1 because it parsed successfully and the available tests used only benign inputs that produced the expected output. However, L2 still flagged the reachable unsafe shell execution, and L3 assigned a high score because a user-controlled source co-occurred with a dangerous sink matching the task CWE. Manual review therefore confirmed the patch as a silent failure: it appeared successful under functional evaluation but preserved the original injection path.

2.4.3. Manual Review and Confirmation Criteria

The L2/L3 pipeline flags candidates automatically, but static analysis can produce irrelevant or non-actionable warnings. The first author conducted the initial manual review of every flagged case following a structured coding procedure. To reduce individual bias, a subset of cases was independently reviewed by another author as part of the reliability assessment described in Section 2.5.3.

Review Process and Consistency Mechanisms To maintain consistency across the entire dataset, we applied an iterative review protocol:

- (i) **Standardized Annotation:** The first author examined the generated patch, L2/L3 findings, and original task description for each case. Judgments and metadata (e.g., failure type, severity) were recorded using a codebook [28], which strictly defined the admissible values for each dimension.
- (ii) **Iterative Alignment (Within-iteration):** The review was conducted iteration-by-iteration. At the end of each iteration, iteration-level summaries were compiled to analyze the distribution of judgments and re-examine borderline cases.
- (iii) **Retrospective Revision (Across iterations):** Following iteration 3, all codes were consolidated to ensure uniformity. If a newly observed case prompted a refinement to the coding scheme, earlier judgments were retrospectively updated. Adjustments were recorded during the coding process to support transparency and consistency.

Table 3

Iteration-based data collection plan.

Iteration	Tasks	Source	Sampling Logic	Coding Activity
Iteration 1	50	SecurityEval	Proportional by CWE	Open Coding
Iteration 2	50	SecurityEval + CVEfixes	Targeted framework–dataset pairs	Axial Coding
Iteration 3	50	SecurityEval + CVEfixes	Underrepresented patterns	Selective Coding
Total: 150 tasks		× 7 agents = 1,050 target traces		

- (iv) **Independent Validation:** To assess inter-rater reliability, another author independently recoded 20% of the corpus (reliability measures reported in Section 2.5.3).

Category Definitions Based on the review, each candidate was assigned to exactly one of the following categories:

- **True Positive (TP):** The patch contains a clear instance of the target vulnerability. The agent either made no attempt to fix it or applied a superficial fix that entirely failed to address the root cause. For example, validating input length while leaving an unsafe call intact.
- **Partial:** The patch contains a visible and relevant mitigation attempt, but the defense remains incomplete and a plausible exploit path is still present in the code. Unlike TP, a defense against the target CWE is visibly attempted; unlike FP, the defense does not fully neutralize the weakness. For instance, the patch blocks one specific bypass while leaving the sink reachable through another path, or constrains a symptom rather than the underlying sink.
- **False Positive (FP):** The patch does not contain a genuine security weakness relevant to the task, even though it was flagged by the pipeline. This typically occurs when the flagged pattern appears only in non-deployable test code, when an effective safeguard already neutralizes the risky operation, or when the static tool matches code that appears dangerous syntactically but is harmless in context.

Both TP and Partial cases are retained in the confirmed corpus. We include Partial cases because they exhibit the same code-level failure patterns as TPs and are relevant for taxonomy construction, which focuses on what the agent did wrong rather than on exploitability severity. In our corpus, Partial cases account for 39 of 170 confirmed failures.

2.5. Data Collection and Analysis

2.5.1. Data Collection

Data collection consisted of two steps: sampling and preprocessing.

1. **Sampling.** We purposively sampled 150 tasks in three iterations of 50 each [29], drawn without replacement from a pool of 1,236 eligible tasks. In this study, an iteration refers to one sampling round followed by agent

execution and preliminary verification-based screening. Each iteration involved: (i) selecting a batch of tasks; (ii) executing the selected agent frameworks on these tasks; (iii) normalizing the outputs into the common trace schema; (iv) screening the resulting traces through the Multi-Level Verification Framework; and (v) reviewing the resulting candidate cases to guide the sampling focus of the next round [30, 31]. The three iterations served different purposes. (i) Iteration 1 sampled from SecurityEval proportionally across CWE categories to obtain broad initial coverage. (ii) Iteration 2 used targeted sampling based on the patterns observed in Iteration 1, especially recurring or ambiguous failure patterns that required further evidence. (iii) Iteration 3 focused on underrepresented open codes and boundary cases to supplement missing failure patterns and refine category coverage. This iterative strategy was intended to support progressive coverage of silent-failure patterns rather than statistical representativeness. Table 3 summarizes the sampling details.

2. **Preprocessing.** To address the inconsistencies in execution trace formats across different frameworks, we normalized all data into a unified schema to enable cross-framework comparison. This step was informed by recent multi-agent failure dataset construction practice, where cross-framework trace analysis requires standardized representations to support consistent annotation and comparison [11]. After normalization, we excluded 20 traces that did not produce valid outputs due to timeouts, runtime crashes, or incomplete generation, leaving 1,030 valid traces. We then evaluated each valid trace through the multi-level verification framework described in Section 2.4.2. Generated patches that failed at L0 or L1 were classified as explicit failures. Those that passed both L0 and L1 but were flagged at L2 or L3 yielded 252 silent failure candidates. After manual review, 170 cases were confirmed as silent failures. These 170 cases form the corpus for the coding procedure described next.

2.5.2. Coding Procedure

We analyzed confirmed cases using an iterative thematic coding process inspired by Grounded Theory [32]. Coding was conducted in three stages, using individual execution traces (one framework’s output per task) as the unit of analysis.

Table 4

Inter-rater agreement across coding dimensions.

(a) D0 triage verdict					(b) D1 axial category					(c) Composite D0xD1								
R1 \ R2	TP	Partial	FP	Total	R1 \ R2	Om.	Inad.	Intro.	FP	Total	R1 \ R2	FP	P-I	P-O	T-I	T-Intro	T-O	Total
TP	19	2	2	23	Om.	8	0	0	1	9	FP	6	1	0	0	0	1	8
Partial	0	3	1	4	Inad.	0	7	0	2	9	P-I	1	2	0	0	0	0	3
FP	1	1	6	8	Intro.	0	2	7	0	9	P-O	0	0	1	0	0	0	1
Total	20	6	9	35	FP	1	1	0	0	2	T-I	1	1	0	4	0	0	6
					Total	9	10	7	3	29	T-Intro	0	1	0	1	7	0	9
											T-O	1	0	0	0	0	7	8
											Total	9	5	1	5	7	8	35

Note: Om. = Omission; Inad. = Inadequacy; Intro. = Introduction; P-I = Partial-Inadequacy; P-O = Partial-Omission; T-I = TP-Inadequacy; T-Intro = TP-Introduction; T-O = TP-Omission.

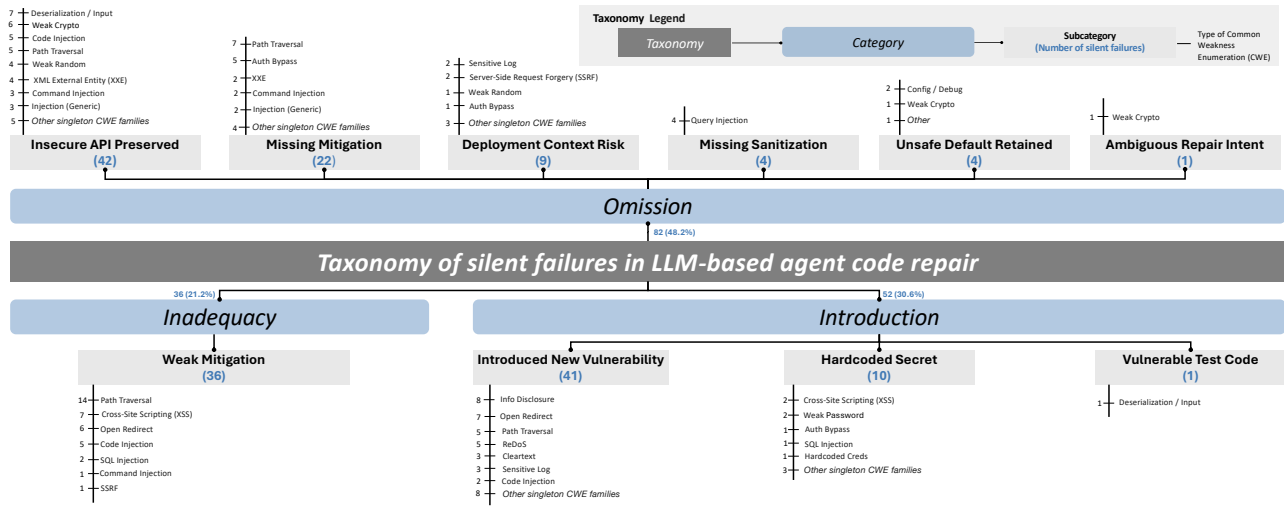


Figure 2: Taxonomy of silent failures in LLM-based agent code repair. The taxonomy classifies failure mechanisms rather than CWE families; therefore, the same CWE family may appear under different open codes when it arises through different mechanisms. "Other singleton CWE families" denotes one-off CWE families grouped only for visualization to avoid over-fragmenting the figure.

1. *Open coding.* The first author examined each confirmed case, including the generated code, L2/L3 findings, and the original task context, and assigned descriptive labels to observed failure patterns. No codes were predefined. Each case was also annotated with a propagation chain, a severity rating reflecting the coder's overall assessment of the weakness (Critical, High, Medium, Low), and candidate detection mechanisms.
2. *Axial coding.* The first author analyzed the initial codes and organized them along four dimensions: failure type, originator role, injection stage, and code location. Relationships between dimensions were examined and documented.
3. *Selective coding.* The first author reviewed and refined the coding results. We identified an overarching theme, security-compliance decoupling, that connects the categories into a unified explanation of silent failures. Categories were merged, split, or dropped based on discussion among all authors.

Throughout this process, each newly coded case was compared against earlier cases in the same category to

check for consistency. We considered coding complete when no new codes emerged in two consecutive batches of 50 traces [30].

2.5.3. Reliability Measures

To ensure the consistency of the qualitative coding process and mitigate individual bias, we performed an Inter-Rater Reliability (IRR) analysis using Cohen's Kappa [33]. A random subset of 35 cases was independently coded by another author (R2), in parallel with the primary coder (R1). Each case was assessed along two coding dimensions: (i) *D0 (triage verdict)*, classifying the case as TP, PARTIAL, or FP; and (ii) *D1 (axial category)*, recording the axial category assigned by a rater, with FP retained as an exclusion label for cases a rater did not treat as a silent failure.

Agreement was evaluated at three levels, as shown in Table 4. First, we measured agreement on the D0 triage verdict, which distinguishes true positives, partial cases, and false positives. Second, we measured agreement on the D1 axial category, which assigns non-false-positive cases to the main silent-failure categories. This analysis included 29 cases because cases coded as FP by both raters were excluded

Table 5

Distribution and exploitability profile of silent failure open codes.

Axial Category	Open Code	Description	<i>n</i>	% in Total	Mean L3
Omission	Insecure API Preserved	Dangerous API retained without secure substitution	42	24.7	16.7
	Missing Mitigation	Required security control entirely absent	22	12.9	5.8
	Deployment Context Risk	Security adequacy depends on deployment context	9	5.3	4.2
	Missing Sanitization	Input sanitization absent at a critical point	4	2.4	2.0
	Unsafe Default Retained	Insecure configuration value left unchanged	4	2.4	1.0
	Ambiguous Repair Intent	Repair intent ambiguous; security effect indeterminate	1	0.6	0.0
	<i>Subtotal</i>		82	48.2	10.7
Inadequacy	Weak Mitigation	Defense attempted but bypassable or incomplete	36	21.2	17.7
	<i>Subtotal</i>		36	21.2	17.7
Introduction	Introduced New Vulnerability	Repair introduced a vulnerability not originally present	41	24.1	6.9
	Hardcoded Secret	Cryptographic secret embedded as string literal	10	5.9	10.4
	Vulnerable Test Code	Test code reintroduced a fixed vulnerability	1	0.6	29.0
	<i>Subtotal</i>		52	30.6	8.0
Total			170	100.0	11.4

from the axial-category calculation. Third, we computed a composite D0×D1 agreement score, which combines the triage decision and the axial category into a single coding decision. We report the composite score as the primary inter-rater reliability metric because it captures both whether a candidate was treated as a confirmed silent failure and how it was categorized.

The primary composite agreement was $\kappa = 0.71$, indicating substantial agreement. The D0 triage verdict and D1 axial category also reached substantial agreement, with $\kappa = 0.63$ and $\kappa = 0.66$, respectively. All disagreements were resolved through discussion among the authors. These discussions were used to clarify ambiguous coding rules, refine category boundaries, and update the final codebook before applying it consistently to the full corpus. In addition, each newly coded case was compared against prior cases in the same category to reduce category drift during the iterative coding process.

3. Results

This section presents the results of our empirical investigation, addressing the three research questions outlined in Section 1: RQ1 on failure taxonomy and prevalence, RQ2 on propagation patterns across agent architectures, and RQ3 on the distribution of silent failures across vulnerability types, code locations, and severity or exploitability levels. Throughout this section, we present axial categories in **boldface**, open codes in Title Case (e.g., Insecure API Preserved), and conceptual terms in *italics*. We report the silent failure taxonomy and prevalence in Section 3.1 (see

Figure 2 and Table 5), propagation patterns across agent architectures in Section 3.2 (see Table 6 and Figure 4), and the distribution of silent failures across vulnerability types, code locations, and severity or exploitability levels in Section 3.3 (see Figure 6 and Table 8).

3.1. Silent Failure (RQ1)

Figure 2 provides the complete taxonomy of silent failures identified in this study. The taxonomy was derived through qualitative coding of 170 confirmed silent failures and consists of three axial categories: Omission, Inadequacy, and Introduction with ten open codes. Table 5 complements the figure by reporting the quantitative distribution and mean L3 exploitability risk score for each open code and category.

The results show that Omission is the most frequently observed category (82/170, 48.2%), followed by Introduction (52/170, 30.6%) and Inadequacy (36/170, 21.2%). A silent failure may exhibit more than one security symptom. However, we assign each case to the dominant failure mechanism, namely the mechanism most directly responsible for the failure to satisfy the security intent of the task. Figure 3 further shows how the open codes differ in both prevalence and exploitability risk.

1. Omission (82/170, 48.2%). Omission refers to cases in which the agent fails to implement a necessary security measure despite the task context indicating that such a measure is required. This is the largest category in the taxonomy of silent failures. We identified and classified six open codes under Omission (see Table 5). Each of them is briefly described below.

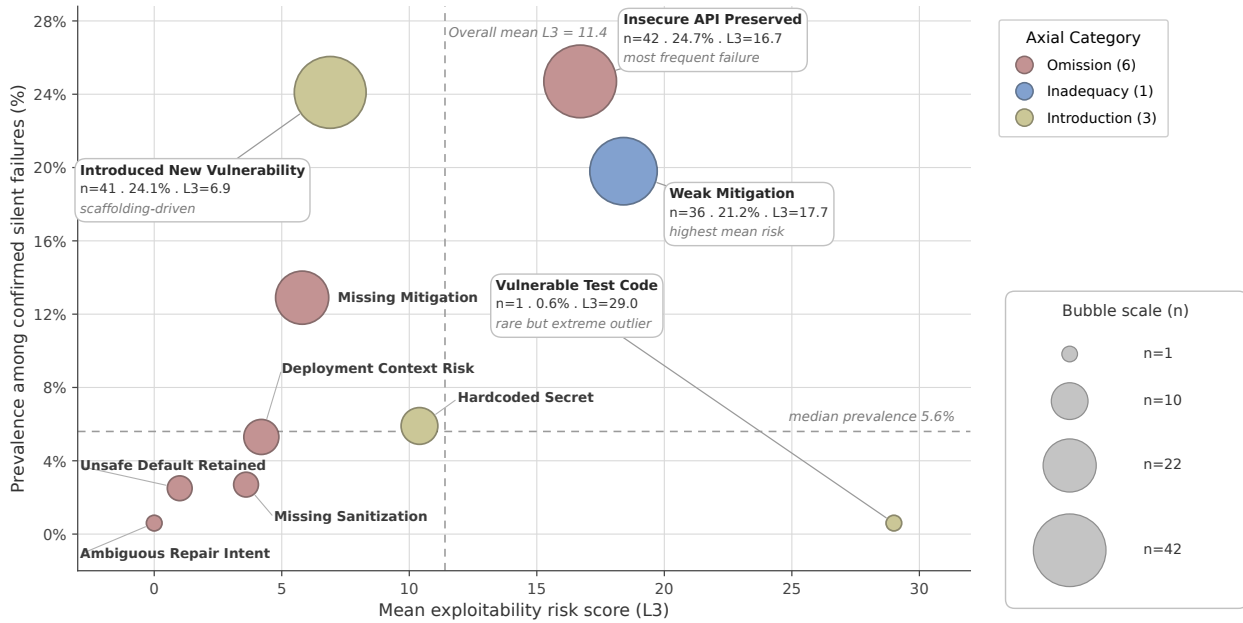


Figure 3: Risk-prevalence landscape of silent failure codes. Each bubble represents one open code; horizontal axis shows mean L3 exploitability risk, vertical axis shows prevalence (%) among the $n = 170$ confirmed silent failures, bubble size encodes the absolute count n , and color encodes the axial category. Reference lines mark the overall mean L3 (11.4) and median prevalence across codes.

- **Insecure API Preserved (42, 24.7%):** The agent retains a dangerous API call from the input context without substituting a secure alternative. This is the dominant code within Omission and the most frequent code in the entire corpus. We observed that agents often applied cosmetic adjustments such as renaming variables or adding length checks, while leaving the vulnerable call intact.

- **Missing Mitigation (22, 12.9%):** The agent omits an entire security control that the task requires but the existing code does not suggest. This code appeared predominantly in CVEfixes tasks, which more frequently require adding new security logic rather than substituting an API. Unlike Insecure API Preserved, which reflects insecure pattern reproduction, Missing Mitigation captures cases where the required defense must be inferred from task intent rather than copied from the input context.

- **Deployment Context Risk (9, 5.3%):** The generated code may appear secure in isolation, but its security adequacy depends on deployment-time conditions that the task does not specify. The agent leaves these conditions unaddressed, transferring the security burden to downstream operators. Examples include hardcoded permissive CORS policies and unverified trust assumptions about upstream input sources.

- **Missing Sanitization (4, 2.4%):** Input sanitization is absent at a critical point along the data flow. This code differs from Missing Mitigation in that the required defense is specifically input validation or escaping at an identifiable sink. The small count reflects how seldom this isolated

pattern appeared without overlapping with other Omission codes.

- **Unsafe Default Retained (4, 2.4%):** An insecure configuration value present in the input is left unchanged in the generated code. Observed instances include retained debug flags and default credentials in authentication-related tasks.

- **Ambiguous Repair Intent (1, 0.6%):** The repair intent is ambiguous and the security effect cannot be determined from the generated artifact alone. We retained this code to mark a boundary case rather than discard the trace, but it represents an edge case in our corpus.

2. Inadequacy (36/170, 21.2%). Inadequacy refers to cases in which the agent appears to recognize the security requirement and attempts a defense, but the resulting mechanism remains incomplete or bypassable. This category differs from Omission in that a security-relevant action is present in the generated code; what is missing is its sufficiency. We identified a single open code under Inadequacy. It is briefly described below.

- **Weak Mitigation (36, 21.2%):** The agent implements a defense that is partial, bypassable, or built on an incorrect security primitive. Among the open codes occurring in more than one case, Weak Mitigation carries the highest mean L3 exploitability risk (17.7), suggesting that partially implemented defenses are associated with a viable attack surface in the confirmed cases. Cases concentrated in vulnerability types such as path traversal, command injection, and eval-based execution. For example, one agent implemented path traversal prevention using a string-level comparison function rather than a path-aware alternative; another added a

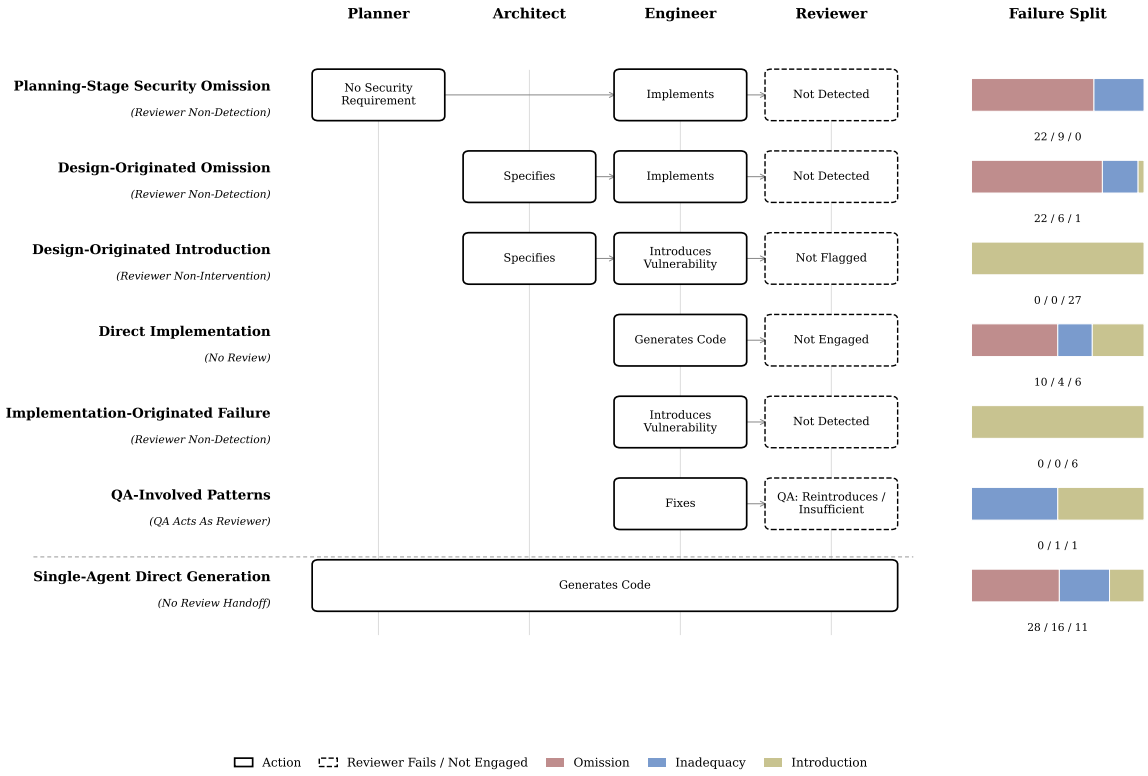


Figure 4: Propagation chain patterns of silent failures by origin, failure mechanism, and review outcome. Color-coded bars on the right show the axial category split for each pattern.

command allowlist but retained an injection-prone execution mode; a third constructed a restricted evaluation sandbox whose filtering rules were too permissive to prevent known escape techniques.

3. Introduction (52/170, 30.6%). Introduction refers to cases in which the repair process itself injects a vulnerability that was not present in the original code. Unlike Omission and Inadequacy, which describe what the agent failed to do, Introduction describes a new defect arising from agent action. We identified and classified three open codes under Introduction. Each of them is briefly described below.

- **Introduced New Vulnerability (41, 24.1%):** The agent generates code that introduces a vulnerability absent from the original input. This code is the dominant pattern within Introduction (41/52, 78.8%). Multi-agent frameworks contributed a higher share, concentrated in ChatDev and MetaGPT. These frameworks also generated additional artifacts, such as configuration files, build scripts, and test scaffolding, where some of these introduced vulnerabilities appeared.

- **Hardcoded Secret (10, 5.9%):** The agent embeds cryptographic secrets (e.g., signing keys, tokens, passwords) as string literals in the generated code. The surrounding authentication or session logic is often otherwise correct, which makes this pattern particularly easy to overlook in functional review.

- **Vulnerable Test Code (1, 0.6%):** The agent introduces a vulnerability through generated test code rather than production code. Although rare, this case stands out with an L3 score of 29.0, far above the Introduction category mean of 8.0, since the test artifact reintroduced a previously fixed vulnerability into the project workspace.

Taken together, the three axial categories share a common characteristic: in every confirmed case, the agent produced code that satisfied L0 and L1 verification criteria yet failed to achieve the security intent of the task. We refer to this recurring pattern as *security-compliance decoupling*: the observed disconnect between an agent’s externally observable compliance signals and the actual security properties of the generated artifact. The three categories correspond to distinct points at which this disconnect appears: Omission involves a gap between the task and appropriate API selection; Inadequacy involves a gap between identifying the security requirement and implementing a complete defense; Introduction involves a gap between addressing the primary task and managing the security context of all generated artifacts.

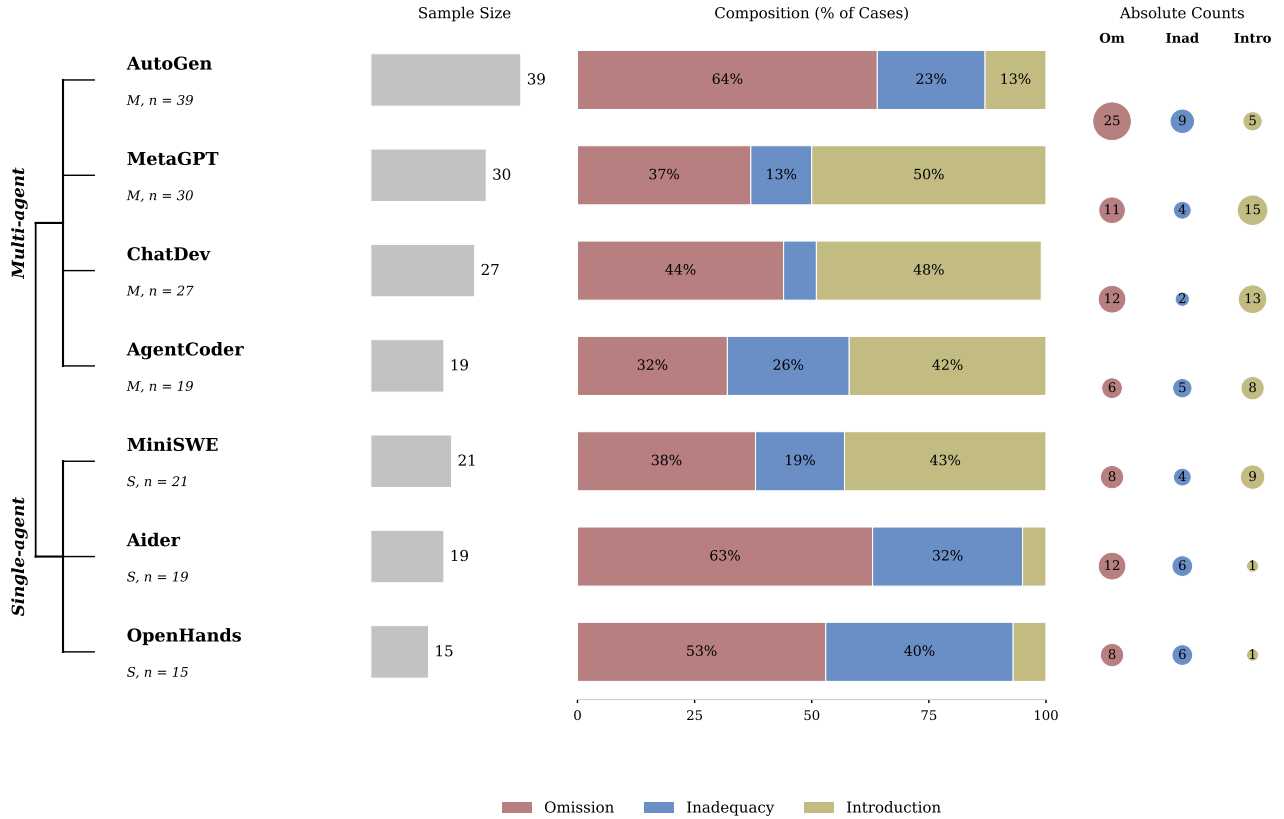


Figure 5: Distribution of silent failure categories across agent frameworks.

Key Findings from RQ1

We identified three recurring mechanisms of silent failures: Omission, Introduction, and Inadequacy. These mechanisms share a common pattern of *security-compliance decoupling*, where generated patches appear to satisfy functional requirements but fail to preserve security properties. We further derived ten open codes to capture the specific failure patterns, with Insecure API Preserved and Introduced New Vulnerability being the most frequent. Among the three mechanisms, Inadequacy showed the highest mean exploitability risk, suggesting that partially implemented defenses may be especially dangerous because they create an appearance of security while remaining bypassable.

3.2. Propagation Patterns Across Agent Architectures (RQ2)

We use the same three axial categories as in Section 3.1: Omission, Inadequacy, and Introduction, denoting absent defenses, incomplete defenses, and newly introduced vulnerabilities, respectively. The propagation patterns are summarized in Table 6, and Figure 4 illustrates the operational chain of each pattern by origin stage, failure mechanism, and review outcome. The architectural comparison is reported in

Table 7, and the distribution of categories across frameworks is shown in Figure 5. Each aspect is discussed below.

Originator attribution was assigned during manual coding (see Section 2.5). For each confirmed case, coders inspected the per-role trace outputs and identified the earliest role whose output made the security deficiency observable in the trajectory. When the deficiency first appeared in a planning or design artifact, the corresponding Planner or Architect role was coded as the originator; when it first appeared in generated code, the Engineer role was coded as the originator. Disagreements were resolved through discussion. For instance, in one path traversal task, the Planner decomposed the request into file-reading and response-formatting steps but omitted the need to constrain user-provided paths to a safe base directory. The Engineer implemented the under-specified plan by joining user input with a base path without canonicalization, and the Reviewer approved the output after checking functional completeness rather than path-handling security. Because the first observable deficiency appeared in the planning artifact and persisted through implementation and review, we coded the Planner as the originator and the propagation pattern as a planning-stage omission cascade.

Among multi-agent systems, the three most frequent patterns all involve a Reviewer role that does not intercept the security deficiency. The most common is the planning-stage omission cascade, in which the Planner decomposes

Table 6
Propagation chain patterns of silent failures.

Propagation Pattern	Om	Inad	Intro
<i>Multi-Agent</i>			
Planning-Stage Omission Cascade	22	9	0
Design-Originated Omission	22	6	1
Design-Originated Introduction	0	0	27
Direct Implementation (No Review)	10	4	6
QA-Involved Patterns	0	1	1
Implementation-Originated	0	0	6
<i>Single-Agent</i>			
Single-Agent Direct Generation	28	16	11
Total	82	36	52

Table 7
Axial category distribution by architecture type.

	Om.	Inad.	Intro.	Total
Multi-agent	54 (47.0%)	20 (17.4%)	41 (35.7%)	115
Single-agent	28 (50.9%)	16 (29.1%)	11 (20.0%)	55
Total	82	36	52	170

the task without including security requirements, the Engineer implements code consistent with the underspecified plan, and the Reviewer does not flag the issue. Following closely is design-originated omission, where the Architect specifies a design that the Engineer implements and the Reviewer evaluates without detecting the deficiency. The third is design-originated introduction; ChatDev and MetaGPT accounted for all instances of this pattern, as their pipelines produce configuration templates and scaffolding that expand the attack surface.

A Reviewer role was active in most multi-agent cases. In the confirmed cases we reviewed, the Reviewer rarely flagged the relevant security deficiency before it reached the final output. This pattern appeared across the multi-agent frameworks in our corpus, despite their differing coordination architectures.

Beyond these multi-agent patterns, Table 7 shows distinct failure distributions between the two architectural families. For instance, Introduction failures are more common in multi-agent systems (41/115, 35.7%) than in single-agent ones (11/55, 20.0%), reflecting the additional configuration and scaffolding artifacts their pipelines produce. By contrast, Inadequacy accounts for a higher share of single-agent failures (16/55, 29.1%) than of multi-agent ones (20/115, 17.4%). This indicates that the observed failure profile differs between the two architecture groups: Introduction is more frequent in multi-agent systems, whereas Inadequacy represents a larger share of single-agent failures. However, the proportion of Omission is nearly identical across architectures (54/115, 47.0% and 28/55, 50.9% respectively).

Further, we observed task-level convergence across frameworks. Specifically, on 26 tasks, three or more frameworks

each produced a silent failure that shared the same confirmed CWE; these comparable failures account for 62.9% (107/170) of all confirmed cases. We treated failures as comparable when they occurred on the same task and shared the same confirmed CWE. Four of the tasks were particularly convergent in that six of the seven frameworks produced a silent failure with the same CWE. This shows that a subset of tasks elicited similar silent-failure patterns across otherwise distinct frameworks, rather than isolated framework-specific failures. However, because all experiments used the same base model and related task prompts, this convergence should not be attributed to framework architecture alone. The present design does not isolate model effects from other shared factors, such as task design, prompt content, or dataset-specific vulnerability patterns. We therefore report cross-framework convergence as an empirical pattern in our corpus.

Key Findings from RQ2

We observed that silent failures propagate through three primary multi-agent mechanisms: planning-stage security omission cascades, design-originated omissions, and design-originated introductions. In the multi-agent cases we reviewed, the Reviewer rarely flagged a security deficiency before it reached the final output. We further observed different failure profiles across agent architectures: multi-agent systems produced more INTRODUCTION failures, consistent with their additional artifact generation, whereas single-agent systems produced more INADEQUACY failures, consistent with attempted but incomplete defenses. Finally, similar failures recurred across frameworks on the same tasks, indicating cross-framework convergence within the observed corpus.

3.3. Distribution Across Vulnerability Types, Code Locations, and Risk Levels (RQ3)

The distribution across CWE families is shown in Figure 6, the distribution across code locations is reported in Table 8, and the severity and risk profiles are reported in Table 9. These three views capture, respectively, what kind of vulnerability is affected, where in the code the failure resides, and how severe and exploitable it is. Across all three, the axial categories align with distinct profiles rather than spreading evenly, as detailed below.

Specifically, as Figure 6 illustrates, Omission accounts for the large majority of cases in families where the fix involves substituting a known insecure API with a secure alternative: weak cryptography (8/9, 88.9%), deserialization (7/8, 87.5%), XXE (7/8, 87.5%), and command injection (5/6, 83.3%). These families share a common trait: the insecure pattern is visible in the code, and its secure counterpart is well-documented. Conversely, Inadequacy clusters in families whose fix must cover multiple input cases or bypass routes, including XSS and path traversal. Finally,

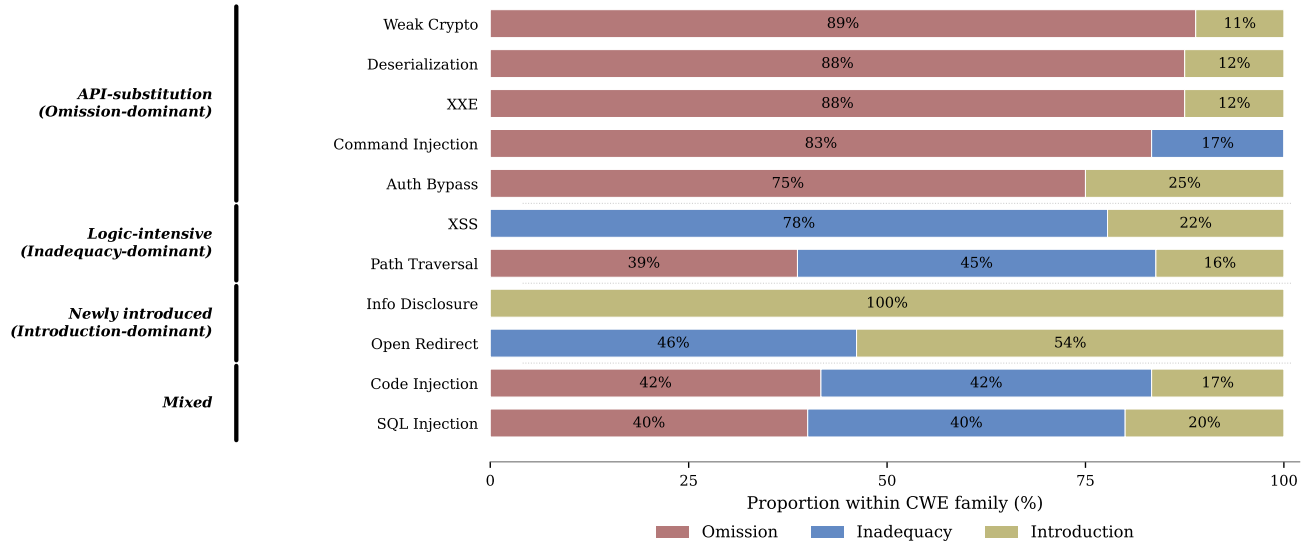


Figure 6: Silent failure category distribution by CWE family.

Table 8

Silent failure distribution by code location and axial category.

Code Location	Om.	Inad.	Intro.	Total	In Total
Input Handling	26	33	2	61	35.9
Configuration	5	0	32	37	21.8
API Call	35	0	0	35	20.6
Business Logic	7	3	9	19	11.2
Authentication	2	0	7	9	5.3
Other	7	0	2	9	5.3
Total	82	36	52	170	100.0%

Note. The *In Total* column shows each code location's share of the 170-case corpus, with bar length proportional to that share. Three locations (Input handling, Configuration, and API call) together account for over three-quarters of all confirmed silent failures, and each is the dominant location for one axial category respectively (Inadequacy, Introduction, and Omission).

Introduction is associated with newly created flaws, such as information disclosure and open redirect.

Besides vulnerability family, code location provides a complementary view of where silent failures reside. Table 8 presents the distribution across code locations. Omission is most frequent in API calls and input handling, corresponding to cases where the agent retained a dangerous function call or omitted required validation logic. Inadequacy is predominantly located in input handling (33/36), reflecting cases where agents attempted validation or sanitization but implemented it incorrectly. Introduction is highly concentrated in configuration and business logic, corresponding to vulnerabilities added through scaffolding generation or auxiliary functionality. The alignment between code location and axial category suggests that the taxonomy dimensions capture relatively distinct aspects of the failure phenomenon.

Table 9

Severity distribution and L3 risk by axial category.

	Om.	Inad.	Intro.	Total
Severity				
Critical	17	5	0	22
High	31	17	5	53
Medium	34	14	46	94
Low	0	0	1	1
L3 Risk				
Mean	10.7	17.7	8.0	11.4
Max	60.8	100.0	58.0	100.0

Note. Color intensity encodes magnitude per row. Mean L3 and Max L3 denote the mean and maximum exploitability risk score under L3 verification, respectively.

The severity and exploitability profiles further differentiate the three categories. Severity follows the rating assigned during the coding procedure (see Section 2.5.2), and the L3 risk score is the exploitability estimate produced by our L3 procedure (see Section 2.4.2). With these definitions, Table 9 reports the severity distribution and L3 risk by category. Omission contains the largest number of critical-severity cases, predominantly involving unrestricted code execution and insecure deserialization. Introduction has the lowest severity profile, with most cases rated as medium (46/52, 88.5%), reflecting that introduced vulnerabilities tend to be configuration-level issues rather than direct execution paths. However, Inadequacy shows the highest mean L3 risk (17.7), above Omission (10.7) and Introduction (8.0). Many Inadequacy cases occur in logic-intensive families such as XSS and path traversal, where partial defenses can leave bypass routes open.

Overall, these distributions reveal three cross-dimensional relationships. First, CWE family is associated with axial category: API-substitution vulnerabilities such as weak

cryptography and deserialization map predominantly to Omission, logic-intensive vulnerabilities such as path traversal and XSS map to Inadequacy, and vulnerability types such as information disclosure and open redirect map to Introduction. This association indicates that vulnerability type helps characterize how silent failures manifest in the observed corpus. Second, each axial category was concentrated in a characteristic code location: Omission concentrates in API calls, Inadequacy in input handling, and Introduction in configuration artifacts. This separation is consistent with the taxonomy dimensions capturing distinct aspects of the failure phenomenon. Finally, severity does not follow a simple gradient across the three categories; instead, Inadequacy is associated with the highest exploitation risk despite representing attempted rather than absent defenses.

Key Findings from RQ3

We found that different vulnerability contexts are associated with different types of silent failures in our corpus. API-substitution vulnerabilities are predominantly associated with Omission, logic-intensive vulnerabilities with Inadequacy, and vulnerabilities introduced during repair with Introduction. Among these categories, Inadequacy exhibits the highest estimated exploitability risk. These results indicate that CWE family and code location help characterize the types of silent failures observed in this study.

4. Discussion

This section discusses the key findings for each research question and outlines their implications for research and practice. It examines why patches that pass functional validation can remain insecure (RQ1), why current evaluation and review practices do not surface these cases (RQ2), and how the observed failure patterns can inform more targeted verification (RQ3).

4.1. Security-Compliance Decoupling as a Distinct Failure Phenomenon (RQ1)

Silent failures as a structurally distinct class of defects. The security-compliance decoupling we identified in our results (see Section 3.1) is structurally distinct from conventional software defects. In an ordinary bug, the code fails to produce expected behavior, and this failure is in principle detectable through functional testing. In a silent failure, by contrast, the code not only runs but produces the expected result; the deficiency lies in a non-functional dimension, namely the security property of the output, that the evaluation oracle was not designed to assess. This distinction matters because it implies that silent failures are unlikely to be eliminated by improving test coverage alone. Instead, they appear to require evaluation criteria that extend beyond functional correctness into security semantics.

This observation aligns with a long-standing finding in automated program repair, namely that plausible patches are

not necessarily correct [1]. The cases we identify, however, go a step further: the patches are not merely plausible but also pass the available functional checks, yet remain insecure at the semantic level. As LLM-based agents become more capable of satisfying increasingly demanding test cases, this gap between functional adequacy and security correctness may widen rather than narrow.

The two prominent open codes reflect different reasoning gaps. At the axial level, the cases concentrate in a small part of the taxonomy: among the ten open codes, the two prominent codes, Insecure API Preserved (42) and Weak Mitigation (36), together account for nearly half of the corpus (78/170, 45.9%). This concentration suggests that security-compliance decoupling is driven by a few recurring failure mechanisms rather than by many isolated factors.

Of the two, Insecure API Preserved is the more prevalent (42 cases) and illustrates the decoupling in its most direct form: the agent completes the functional task while leaving in place a dangerous API call that was already present in the input context. One plausible explanation is insecure pattern reproduction: the agent appears to treat the surrounding code as authoritative and edits around the insecure call rather than replacing it, particularly when that call is not the direct subject of a failing test. As a result, the inherited insecure construct remains in the repaired code, even though the task itself calls for a security fix.

Weak Mitigation, by contrast, reflects a different dynamic, an attempted but incomplete defense. In our taxonomy, Weak Mitigation is the sole open code under Inadequacy, and among the recurring open codes it has the highest mean exploitability score ($L3 = 17.7$), which challenges the intuition that any defense is better than none. A plausible reading is that this gap lies between recognizing a vulnerability and implementing a robust defense: producing a partial defense requires the agent to recognize the vulnerability class, but not necessarily to reason about the conditions under which the defense actually holds. For example, the agent may recognize that an input should be constrained, yet implement a check that known bypass strategies can evade. Consequently, such a visible control can create an appearance of safety that the underlying code does not support.

The remaining open codes further indicate when the dominant mechanisms change. Missing Mitigation appears when the required security control must be inferred from task intent rather than substituted from the input, which is more demanding than replacing a visibly unsafe API. Deployment Context Risk and Unsafe Default Retained suggest that agents tend to treat deployment assumptions as peripheral rather than as part of the repair obligation, while Hardcoded Secret points to a related tendency to produce runnable examples rather than production-safe security artifacts. Missing Sanitization and Ambiguous Repair Intent appear in only a few cases and mark boundary conditions rather than recurring mechanisms. Taken together, these lower-frequency codes suggest that future agent design may need to distinguish API-substitution tasks from tasks that

instead require new security reasoning, deployment-context reasoning, or secure-default generation.

These two dominant mechanisms are consistent with prior evidence on the structural limitations of LLMs. The tendency to preserve insecure context aligns with reported difficulties of agentic systems in reliably interpreting user intent [34, 35]: the observed behavior is consistent with satisfying the literal task while failing to address its broader security purpose. Likewise, the incompleteness of attempted defenses aligns with evidence that LLM-based vulnerability repair depends on context when applied to real-world vulnerabilities [36], and with the long-standing automated program repair finding that a test-passing patch may mask rather than resolve the underlying defect [1].

Different decoupling modes may call for different remediation strategies. The three axial categories appear to represent qualitatively different points at which the decoupling occurs. Omission involves a gap between understanding the task and selecting an appropriate API or control. Inadequacy involves a gap between identifying the security requirement and implementing a complete defense. Introduction involves a gap between addressing the primary task and managing the security context of all generated artifacts. The heterogeneity of these failure modes is consistent with the MAST taxonomy, which likewise finds that failures in multi-agent systems fall into qualitatively distinct categories rather than a single defect type [11]. In the verification literature, hybrid pipelines that combine LLM-based generation with static analysis or formal methods [20, 37] similarly indicate that a single class of checks may not be sufficient for all deficiency types. If these distinctions hold, no single mitigation strategy would suffice: Omission may respond to API substitution rules or secure-by-default libraries; Inadequacy may require deeper security reasoning or defense completeness verification; Introduction may require extending the verification boundary to all artifacts generated during the repair, rather than only the prompted target.

Implications. Taken together, these results suggest that security deficiencies should not be treated as a monolithic category, and that verification pipelines may benefit from mode-specific checks matched to the expected failure profile of a given task. For researchers, the dominance of a single, mechanically simple code indicates that a substantial portion of silent failures may be addressable through targeted, rule-based interventions, while the risk concentration in Weak Mitigation points to completeness-oriented verification, such as differential testing against known bypasses, as a priority for evaluation. For agent developers, safe-API substitution rules or curated secure-default mappings, applied before a patch is accepted, could remove a large share of the most common failures. For security engineers, agent-generated controls should be evaluated for completeness against known bypass techniques, not merely for presence, because a bypassable defense can lead downstream reviewers to assume that the issue has been closed and may therefore carry greater practical risk than an absent one.

4.2. Why Current Assurance Pipelines Miss Silent Failures (RQ2)

Current evaluation paradigms may leave security-relevant gaps. Industry-standard benchmarks such as SWE-bench [13] and HumanEval [14] treat test-passing as the primary success criterion. Under these criteria, every confirmed case in our corpus would have been classified as a successful repair. This does not diminish the value of functional benchmarks as operational baselines, but it suggests that benchmark-reported success rates for security-sensitive tasks may overestimate actual repair quality. Complementing functional checks with security-oriented verification appears necessary to surface deficiencies that tests alone were not designed to capture. This concern is consistent with recent APR surveys, which note that current benchmarks offer few metrics for vulnerabilities introduced during repair [38].

Failures often originate early and survive to the final patch. The propagation analysis suggests that reviewer failure is only one part of a broader chain-level problem. In many cases, the deficiency originated upstream and then passed through subsequent roles without security-focused re-evaluation. Although reviewer agents rarely intercepted confirmed deficiencies in our corpus, treating this solely as a reviewer limitation would understate the role of earlier pipeline stages, because the deficiency was already present before review, and later roles tended to assess task progress rather than reconsider the security adequacy of prior decisions. A plausible interpretation is that agent handoffs preserve functional intent more reliably than security intent: once a security control is omitted or weakened at design or implementation time, subsequent roles rarely reconstruct the missing requirement in our cases.

This lifecycle pattern relates closely to prior work on failure dynamics in multi-agent systems. Studies of cascading failures and problem drift describe how errors compound across interaction steps [39], and attribution frameworks such as AgenTracer localize fault-inducing steps within agent trajectories [18]. These approaches, however, share a common limitation: they often assume an observable failure signal, such as an explicit error or a failing test, to define and trace the fault. Our propagation analysis, by contrast, suggests that similar cross-step dynamics can also arise when no such signal is present. This extends the study of multi-agent failure to latent deficiencies that remain invisible to functional evaluation.

Convergence patterns are consistent with shared experimental factors, while failure profiles differ across architectures. Convergence is a common pattern in our corpus: 107 of the 170 confirmed failures fall within convergent cases, in which 26 tasks triggered comparable failures across three or more frameworks. Since the frameworks differ in orchestration and coordination architecture, this recurrence is consistent with shared factors across the experimental setup, including the base model, prompt content, task design, and recurring vulnerability patterns. This reading is further supported by the earlier observation that agents across the examined architectures tended to reproduce insecure APIs

present in the task context. We do not, however, attribute the convergence to the model alone: because our design fixes the base model and varies the frameworks, it cannot separate model-level effects from shared factors such as prompt content, dataset characteristics, and recurring vulnerability patterns.

Architecture is nevertheless associated with differences in how these failures manifest. The orchestration of each system appears to be associated with how shared failure patterns surface in the generated artifacts: multi-agent systems, which involve more inter-agent interaction and additional artifact generation, exhibited more Introduction failures, whereas single-agent systems exhibited more Inadequacy failures, consistent with attempted but incomplete defenses.

This pattern extends prior evidence on the reliability of LLM-based repair. Earlier work reports that weaknesses recur across prompts and scenarios rather than as isolated, framework-specific errors [36]; our results carry this observation from individual models to agentic repair systems. They suggest that, as long as these recurring security limitations remain unaddressed, similar insecure behavior can recur even under different coordination mechanisms.

Implications. These findings imply that security assurance for agentic repair should be evaluated and designed beyond functional pass rates. For benchmark designers, security-oriented criteria need to be reported alongside functional outcomes so that test-passing is not mistaken for successful security repair. For system designers, assurance mechanisms should move upstream: because silent failures can originate during planning, design, or implementation and persist through subsequent handoffs, tool-supported checks should be deployed where defects first arise rather than only at the final review stage. For researchers, the convergent subset identified in this study offers a useful basis for controlled experiments that vary base models under fixed architectures, or architectures under fixed models, to separate model-level vulnerabilities from architecture-level effects.

4.3. Structured Failure Patterns as a Basis for Targeted Verification (RQ3)

Vulnerability family and code location jointly indicate the expected failure category. The observed association between CWE family and axial category suggests that vulnerability type may help anticipate the type of silent failure an agent is likely to produce. API-substitution vulnerabilities were predominantly associated with Omission; logic-intensive vulnerabilities with Inadequacy; and vulnerabilities introduced during repair with Introduction.

A parallel pattern appears at the level of code location: Omission is concentrated in API calls, Inadequacy in input-handling logic, and Introduction in configuration artifacts. A plausible explanation is that each category's gap surfaces where its work happens. An unsubstituted dangerous call remains visible at the call site; an incomplete defense appears in the input-handling code intended to enforce it; and

a newly introduced weakness often appears in configuration or auxiliary artifacts generated by the pipeline. Both associations are consistent with prior evidence that LLM-based repair performs more reliably on well-specified, pattern-like fixes than on vulnerabilities whose correct handling depends on context [36].

Multi-agent pipelines can introduce vulnerabilities outside the originally targeted patch scope. The co-occurrence of multi-agent architecture, the Introduction category, and configuration-level locations is one of the more distinctive patterns in our data. During execution, multi-agent frameworks may emit scaffolding beyond the requested patch, such as build files, container definitions, runtime settings, and other auxiliary artifacts. These artifacts can contain weaknesses that were absent from the original task. Two factors make such weaknesses difficult to detect: they are not the explicit target of the prompt, so functional tests may not exercise them, and pipeline roles are typically not assigned to review every generated artifact for security. Existing failure taxonomies for multi-agent systems [11] and vulnerability classifications such as CWE help characterize failures or vulnerabilities once they are observed, but they do not address the possibility that the analysis boundary itself may be too narrow. Our results extend this picture by showing that the set of security-relevant artifacts can grow with pipeline complexity.

Silent failures are not confined to low-severity issues. The severity distribution shows that security-compliance decoupling is not limited to minor weaknesses: 22 of the 170 confirmed cases were rated as critical severity, yet they remained undetected by functional testing in the same way as lower-severity cases. This result indicates that functional success does not provide a severity-sensitive assurance signal, because a patch may pass all available tests while still preserving or introducing high-impact vulnerabilities. This observation aligns with arguments in the verification literature for security-oriented test suites that assess repairs beyond functional correctness [19, 40].

Implications. For practitioners, the family and location associations indicate that verification effort can be routed rather than applied uniformly: rule-based API checks at call sites for API-substitution families, deeper logic analysis on input-handling code for logic-intensive families, and scanning that covers generated configuration and auxiliary artifacts for newly introduced vulnerability types. Moreover, because multi-agent pipelines can introduce weaknesses outside the prompted target, vulnerability analysis of agent output should enumerate and scan every artifact the pipeline produces, rather than only the file or component named in the task prompt. For reviewers, the presence of critical-severity cases among silent failures argues against taking a passing test suite as evidence of a secure repair or of acceptable residual risk; independent security verification should therefore precede any decision to trust an agent-generated patch. For researchers, how the set of at-risk artifacts scales with pipeline complexity remains unquantified and could

be measured directly or through iterative re-analysis of the patched code.

5. Threats to Validity

We organize threats to validity following the standard validity categories in empirical software engineering [41].

5.1. Construct Validity

Our operational definition identifies silent failures as patches that satisfy both syntactic and functional verification yet are flagged by static security analysis or exploitability assessment. This definition focuses on security-relevant silent failures. Silent failures involving performance, privacy, maintainability, usability, reliability under load, or other non-security qualities are outside the scope of this study, and our counts should not be read as estimates of failures in those dimensions. Similarly, security properties requiring dynamic analysis, runtime monitoring, environmental configuration, or domain-specific knowledge may not be captured by our verification framework.

Our taxonomy was derived from all 170 confirmed cases, which include 39 (22.9%) classified as Partial alongside 131 (77.1%) true positives. Including Partial cases may affect precision, because a Partial verdict reflects less direct evidence of exploitability than a true-positive verdict. As a sensitivity check, we re-examined the taxonomy using only the 131 true positives. All three axial categories and seven of the ten open codes remain represented in this subset. The three omitted codes occurred only as Partial cases: Deployment Context Risk (9 cases), Ambiguous Repair Intent (1 case), and Vulnerable Test Code (1 case). This is expected, since their exploitability could not be confirmed from static evidence alone, which is the same property that placed them in the Partial category. Although this restriction reduced counts across all three categories, the main taxonomy structure remained stable.

The distinction between Omission and Inadequacy depends on whether the agent made any attempt at mitigation. Because the two categories are reported separately, misclassifying boundary cases would shift their relative sizes. We used any observable mitigation attempt in the trace as the deciding criterion and resolved the small number of boundary cases through discussion between coders.

5.2. Internal Validity

The L2 verification level relies on Bandit, which is subject to both false positives and false negatives. False positives were addressed through manual review, which excluded 82 candidates from the confirmed corpus. False negatives are inherently difficult to quantify; the confirmed corpus should be interpreted as a conservative estimate of the silent failures detectable by our multi-level verification framework (see Section 2.4.2), not as an estimate of all possible silent failures in agentic code repair.

Attributing a silent failure to a specific agent role and pipeline stage requires interpretive judgment, particularly where role boundaries are ambiguous in multi-agent traces.

Because originator attribution was derived from qualitative trace analysis, alternative interpretations of complex agent interactions could lead to different role assignments. This threat may influence the reported originator distributions and the propagation-chain frequencies. To mitigate this threat, we relied on observable trace evidence, predefined coding rules, and consensus-based discussion of ambiguous cases.

A related threat concerns the classification of propagation patterns. Patterns such as planning-stage omission, design-originated omission, design-originated introduction, and implementation-originated failure were derived through qualitative analysis of the traces, and some cases could reasonably be assigned to more than one pattern, especially when underspecified requirements and implementation-level omissions occurred in the same trace. Because the frequency of each pattern feeds into our findings, ambiguous assignments could shift the reported pattern distribution. To reduce this threat, we recorded the trace evidence behind each assignment, compared each newly coded case against earlier cases in the same pattern to limit category drift, and discussed boundary cases jointly before assigning a single final pattern.

The L3 exploitability score is a verification level separate from L2. It relies on predefined signal weights, bounded safe-practice discounts, and a fixed candidate threshold (see Section 2.4.2). The fixed threshold therefore affects which cases are judged as silent failure candidates. For example, a true silent failure scoring below the threshold would not enter manual review and would be missed. The score magnitude, in turn, affects the reported exploitability patterns, so the weighting and discount choices, together with any imprecision in the pattern matching of sources and sinks, contribute to these patterns and to the results that build on them.

To reduce this threat, every candidate above the threshold was manually reviewed before confirmation, so the threshold acts as a screening sensitivity parameter rather than a final classification boundary, and over-inclusion at screening is removed by review. We also applied the same scoring procedure uniformly across all frameworks, so comparative claims rely on the ordering of scores rather than their absolute values.

5.3. External Validity

Both datasets are Python-focused and may not represent the full diversity of security-sensitive repair tasks encountered in practice. Tasks involving multi-file repositories, complex dependency chains, or language-specific security idioms are underrepresented. We therefore limit our claims to Python repair tasks, and do not generalize to other languages or to repository-scale repair.

The seven frameworks (four multi-agent, three single-agent) span a range of coordination architectures, but the landscape of agent systems is evolving rapidly. Our sample of seven frameworks may therefore not generalize to proprietary systems. Despite this architectural diversity, because all frameworks were instantiated using GPT-4o-mini, some

observed failure patterns may reflect characteristics of the underlying model rather than the framework architecture itself. The cross-framework convergence we observed sharpens this concern: it is consistent with shared model-related factors, but it may also be influenced by prompt templates, the characteristics of the two benchmarks, the structure of the repair tasks, recurring vulnerability patterns, or other experimental factors. We therefore treat the source of this convergence as an open question rather than evidence of a single cause, and we cannot fully separate model-specific effects from framework-specific ones. Future studies should replicate the analysis using additional foundation models.

5.4. Conclusion Validity

Conclusion validity concerns whether our results provide support for the conclusions we draw. In this study, the analysis is primarily qualitative and descriptive, so the conclusions are based on repeated patterns and frequency distributions within the confirmed corpus. Several design choices support these conclusions. First, all cases were processed through the same multi-level verification and manual review procedure before entering the confirmed corpus. Second, each confirmed case was assigned to one dominant axial category, which makes the category-level distributions mutually exclusive. Third, the coding criteria were applied consistently across iterations, with boundary cases discussed before final assignment. Lastly, comparisons across frameworks are based on executions conducted under a shared model configuration, timeout, and verification pipeline. These choices reduce threats to conclusion validity. The reported counts are still corpus-dependent, and small differences between adjacent categories, frameworks, or open codes should not be interpreted as stable rankings. We therefore report raw counts together with percentages, describe the results as patterns observed in our corpus, and avoid drawing causal or population-level conclusions from descriptive differences alone.

5.5. Reliability

Qualitative coding was performed by the first author across all three iterations. A random sample of 35 cases was independently coded by another author, yielding a composite Cohen's κ of 0.71. This composite requires agreement on both the manual review label (TP, PARTIAL, or FP) and the axial category (Omission, Inadequacy, or Introduction), with per-dimension values of 0.63 and 0.66 respectively; the procedure is detailed in Section 2.5.3. Disagreements were resolved through structured consensus discussion.

The verification pipeline is deterministic given the same input. Framework execution involves stochastic LLM inference, meaning that exact trace reproduction is not guaranteed.

6. Related Work

To reduce the risk of overlooking relevant studies, we searched seven major digital libraries: Google Scholar, IEEE Xplore, the ACM Digital Library, ScienceDirect, Scopus,

Web of Science, and SpringerLink. To the best of our knowledge, we found no prior work that traces how patches that pass available functional checks but remain insecure originate in agent roles and propagate through agentic repair pipelines. We discuss the existing research in four categories: (i) benchmark-based evaluation of LLM agents (Section 6.1), (ii) failure analysis in multi-agent systems (Section 6.2), (iii) the plausible-versus-correct distinction in APR (Section 6.3), and (iv) security verification of generated repairs (Section 6.4). A concluding summary and comparative analysis (Section 6.5) position the present study at the intersection of these lines and justify its scope and contributions.

6.1. Benchmark-Based Evaluation of LLM Agents

Evaluation of LLM-based agentic systems is often driven by benchmarks. SWE-bench has become a widely used benchmark for assessing automated repair, with related efforts adopting similar outcome-focused metrics that measure whether a generated patch passes existing unit tests [13, 42, 43]. These metrics provide a useful operational baseline, but they treat a passing result as success without assessing whether the generated patch satisfies the security intent of the repair task. As a result, a patch that passes existing tests may still preserve the vulnerable behavior in the original code. Outcome-level evaluations therefore offer limited insight into failure mechanisms or non-functional properties of the output. Beyond these outcome metrics, researchers deploying agentic systems in increasingly complex workflows have begun examining broader limitations such as misalignment, coordination breakdowns, and difficulty in reliably interpreting user intent [34, 35, 44]. These perspectives have helped frame the challenge landscape but remain primarily qualitative, with limited empirical analysis of specific failure patterns.

6.2. Failure Analysis in Multi-Agent Systems

A growing body of work has moved toward empirical failure analysis in multi-agent systems. Agentic systems vary in architecture, including single-agent and multi-agent designs with different role structures and coordination patterns, which may influence where failures appear [45]. The MAST taxonomy provides a systematic categorization of observable failure modes [11], while complementary work has formalized dynamics such as cascading failures and problem drift at the system level [39, 46, 47]. These efforts characterize failures that surface as errors or task non-completion, but they do not address failures that give no such outward indication. Attribution frameworks have advanced toward more granular diagnosis: Who&When and AgenTracer introduced fine-grained annotation and counterfactual replay to identify fault-inducing steps within complex trajectories [17, 18]. Their localization, however, depends on an observable failure signal to define the fault being traced. Beyond diagnosis, resilience mechanisms such as challenger roles and courtroom-inspired inter-agent debate have been proposed to improve reliability [48, 49]. These approaches demonstrate the potential of role-based specialization, though they rely

Table 10

Positioning of representative prior work relative to the present study. Y = explicitly addressed; P = partially addressed; – = not a primary focus.

Representative work	Bench.	Taxon.	Prop.	Role attr.	Sec. verif.	Silent fail.	Main gap relative to this study
Outcome benchmarks (e.g., SWE-bench) [13]	Y	–	–	–	–	–	Evaluate whether patches pass existing tests, but do not assess security-semantic correctness or explain latent security failures.
Multi-agent failure taxonomies (e.g., MAST) [11]	P	Y	P	–	–	–	Characterize observable multi-agent failures, but do not cover failures that leave no external signal such as a crash, error, or failed task.
Trajectory attribution frameworks (e.g., AgenTracer) [17, 18]	–	–	Y	Y	–	–	Localize fault-inducing steps in agent trajectories, but require an observable failure signal to define what should be traced.
APR plausible-versus-correct studies [1, 38, 50]	P	P	–	–	P	P	Establish that test-passing patches may be incorrect, but do not analyze security-specific silent failures in role-based agentic repair pipelines.
LLM security repair and verification [20, 27, 36, 51, 52]	P	–	–	–	Y	P	Assess the security of generated repairs at the artifact level, but do not explain how deficiencies originate, propagate, or evade review across agent roles.
This study	Y	Y	Y	Y	Y	Y	Provides an empirical taxonomy of silent failures, analyzes their role-level origins and propagation chains, and verifies security deficiencies that remain hidden after syntactic and functional success.

Dimensions. **Bench.**: benchmark-based evaluation of repair outcomes; **Taxon.**: classification of failure types; **Prop.**: how failures propagate across pipeline stages; **Role attr.**: attribution of a failure to a specific agent role; **Sec. verif.**: verification of the security of generated code beyond functional correctness; **Silent fail.**: coverage of failures that pass syntactic and functional checks yet retain or introduce a vulnerability.

predominantly on explicit failure signals such as crashes, error messages, or task non-completion. Failures that do not produce such signals remain largely outside their scope. This gap motivates the present study.

6.3. APR and the Plausible-versus-Correct Distinction

While the preceding work examines failure dynamics within agentic pipelines, the failures themselves ultimately materialize in repair artifacts whose quality must also be assessed against established standards in Automated Program Repair (APR) and software security. A central concern in APR is the distinction between plausible and correct patches. A plausible patch passes all tests in the validation suite yet may not correct the defect, and can eliminate desirable functionality or even introduce new security vulnerabilities [1]. Recent APR surveys characterize the field as having evolved from template-based and constraint-based repair methods to machine learning, deep learning, and more recently large-language-model-based approaches [27, 38]. In this evaluation-driven setting, the security of the resulting patches has received comparatively limited attention, and current benchmarks offer few metrics for vulnerabilities introduced during repair [38].

This distinction is the conceptual basis for the silent failures we study, narrowed to the security setting. In APR, the plausibility gap concerns whether a test-passing patch actually fixes the target defect. In our setting, the gap concerns a security property that the available functional tests

do not observe: the generated patch passes syntactic and functional validation, but still leaves the target vulnerability unaddressed or introduces a new one. This security-oriented reading of plausibility has begun to appear in repair evaluations: when a patch is counted as plausibly fixed once it passes the project test cases, neural and LLM-based repair models still fail to address many vulnerability types, including cryptographic and request-handling weaknesses [50]. Such cases show that test passage and security correctness can diverge even when the repair target is a known vulnerability. This limitation is also reflected in LLM-based vulnerability repair, where prior evaluations show that models perform reasonably well in controlled settings but remain unreliable when applied to real-world vulnerabilities [36]. However, these studies primarily evaluate individual patches generated by off-the-shelf models and do not examine how such deficiencies arise and propagate across the roles of a multi-agent repair pipeline. Our study addresses this gap.

6.4. Security Verification of Generated Repairs

A separate line of work addresses this gap through security-oriented verification. Established security test suites have been used for assessment beyond functional correctness [19, 40], while hybrid pipelines combine LLM-based generation with formal verification or static analysis [20, 37]. In industrial practice, such verification is increasingly embedded in CI/CD pipelines [53]. Even where such verification is applied, however, generated fixes can satisfy the surface checks while leaving the underlying weakness

in place; in infrastructure-as-code repair, for instance, a notable share of LLM-generated fixes passed configuration checks yet did not resolve the security issue and escaped scanner detection [52]. This indicates that artifact-level verification reduces, but does not remove, the risk of security deficiencies that survive functional and tool-based checks. Adversarial evaluation sharpens this point: under systematic bypass testing, AI-generated patches were found to rely on minimalistic input validation and to remain exploitable through simple attack techniques, in contrast to the more structural corrections in human-written patches [51]. These approaches assess the security of a patch as a finished artifact, but they do not model how the deficiency arose within the repair process, nor which role in a multi-agent pipeline produced it. As a result, security verification and multi-agent repair remain insufficiently connected, with no account of how a vulnerability passes through agent interactions and evades review.

6.5. Conclusive Summary

These complementary lines of work have each advanced substantially but remain only partially connected. Benchmark studies treat test success as the primary quality indicator and do not examine the security semantics of a passing patch. Failure-analysis studies focus on breakdowns that surface as explicit signals and do not account for outputs that appear correct yet contain latent vulnerabilities. APR research establishes the distinction between plausible and correct patches, and recent work shows that LLM-based security repair remains unreliable, but it analyzes individual patches rather than the roles that produce them. Security-verification studies assess patch quality at the artifact level without modeling how the internal role interactions of a multi-agent pipeline contributed to the deficiency. What remains unaddressed is the intersection of these concerns: how patches that pass functional evaluation yet preserve or introduce exploitable vulnerabilities originate within agent role structures, propagate through pipeline stages, and evade review mechanisms. The present study targets this intersection. Table 10 positions our study against representative prior work along these dimensions and illustrates that the representative prior work considered here does not jointly address agent-role attribution, propagation, security verification, and silent-failure coverage.

7. Conclusion

We present an empirical study of silent failures in LLM-based agentic code repair. To investigate this phenomenon, we analyzed 170 confirmed silent failure cases identified from 1,030 valid execution traces produced by seven agent frameworks across two security-focused datasets. The primary contribution of this study is an empirical characterization of these failures along three dimensions: a three-part taxonomy (Omission, Inadequacy, and Introduction) grounded in confirmed cases, an account of how they propagate through agent pipelines, and an analysis of how they distribute across frameworks, vulnerability families, and

code locations. Based on the analysis of the results, the key findings of this study are as follows:

- Our results show that silent failures should be treated as a separate form of security-compliance decoupling rather than as ordinary functional defects alone. The confirmed cases support a three-part structure consisting of Omission, Inadequacy, and Introduction.
- We also observed that current assurance pipelines centered on test passage and LLM-based review were insufficient to detect these failures. At the same time, the recurrence of similar insecure solutions across frameworks is consistent with shared factors in the experimental setup, including the base model, prompts, tasks, and vulnerability patterns, alongside differences in how failures appear across agent architectures.
- Our results show that silent failures exhibit recurring patterns across vulnerability families, code locations, and generated artifact types in our corpus. These patterns suggest that security assurance for agentic repair should move toward targeted verification that covers the full repair scope rather than general post hoc inspection.

These findings have implications for both research and practice. For research, they suggest that success in agentic repair should no longer be defined solely by whether code compiles and passes tests. Future benchmarks and evaluation protocols should assess both functional validation and security-semantic correctness, and should examine base-model effects and orchestration effects through controlled experimental designs. For practice, they indicate the need to extend verification beyond the primary patched file to all generated artifacts and to augment reviewer roles with tool-backed security checks.

Future work should replicate these findings across models, languages, and tool-supported agent designs, and test whether targeted verification strategies meaningfully reduce silent failure rates in deployment settings. Another direction is to examine agent execution traces to better explain how these silent failures arise, including where security considerations are weakened or dropped during the repair pipeline. This line of work may help shift assurance from post hoc detection toward earlier intervention before patches are accepted.

CRedit authorship contribution statement

Wenji Bai: Conceptualization, Methodology, Software, Formal analysis, Investigation, Data curation, Writing - original draft, Visualization. **Muhammad Waseem:** Conceptualization, Methodology, Validation, Review, Editing, Supervision, Project administration. **Zeeshan Rasheed:** Formal analysis, Investigation. **Jaakko Peltonen:** Validation, Review, Editing, Supervision. **Pekka Abrahamsson:** Resources, Supervision, Review, Editing.

Acknowledgments

This research is funded as part of Finland's Ministry of Education and Culture's Doctoral Education Pilot under Decision No. VN/3137/2024-OKM-6 (The Finnish Doctoral Program Network in Artificial Intelligence, AI-DOC).

Data availability

Link to our dataset is in the reference [21].

Declaration of AI Assistance

During the preparation of this work, the author(s) used ChatGPT to refine grammar, improve sentence structure, and resolve formatting issues. After utilizing this tool, the author(s) thoroughly reviewed and edited the content as needed, taking full responsibility for the final publication.

References

- [1] Z. Qi, F. Long, S. Achour, M. Rinard, An analysis of patch plausibility and correctness for generate-and-validate patch generation systems, in: Proceedings of the 2015 International Symposium on Software Testing and Analysis (ISSTA), 2015, pp. 24–36.
- [2] N. Perry, M. Srivastava, D. Kumar, D. Boneh, Do users write more insecure code with AI assistants?, in: Proceedings of the 2023 ACM SIGSAC Conference on Computer and Communications Security (CCS), 2023, pp. 2785–2799.
- [3] J. Peng, L. Cui, K. Huang, J. Yang, B. Ray, Cweval: Outcome-driven evaluation on functionality and security of LLM code generation, in: 2025 IEEE/ACM International Workshop on Large Language Models for Code (LLM4Code), IEEE, 2025, pp. 33–40.
- [4] J. Yang, C. E. Jimenez, A. Wettig, K. Lieret, S. Yao, K. Narasimhan, O. Press, SWE-agent: Agent-computer interfaces enable automated software engineering, *Advances in Neural Information Processing Systems* 37 (2024) 50528–50652.
- [5] D. Chen, S. Lin, M. Zeng, D. Zan, J.-G. Wang, A. Cheshkov, J. Sun, H. Yu, G. Dong, A. Aliev, et al., Coder: Issue resolving with multi-agent and task graphs, *arXiv preprint arXiv:2406.01304* (2024).
- [6] I. Bouzenia, P. Devanbu, M. Pradel, RepairAgent: An autonomous, LLM-based agent for program repair, in: Proceedings of the IEEE/ACM 47th International Conference on Software Engineering (ICSE), 2025, pp. 2188–2200. doi:10.1109/ICSE55347.2025.00157.
- [7] C. Qian, W. Liu, H. Liu, N. Chen, Y. Dang, J. Li, C. Yang, W. Chen, Y. Su, X. Cong, et al., Chatdev: Communicative agents for software development, in: Proceedings of the 62nd Annual Meeting of the Association for Computational Linguistics (Volume 1: Long Papers), 2024, pp. 15174–15186.
- [8] S. Hong, M. Zhuge, J. Chen, X. Zheng, Y. Cheng, J. Wang, C. Zhang, Z. Wang, S. K. S. Yau, Z. Lin, et al., MetaGPT: Meta programming for a multi-agent collaborative framework, in: Proceedings of the 12th International Conference on Learning Representations (ICLR), 2024.
- [9] Q. Wu, G. Bansal, J. Zhang, Y. Wu, B. Li, E. Zhu, L. Jiang, X. Zhang, S. Zhang, J. Liu, et al., Autogen: Enabling next-gen LLM applications via multi-agent conversations, in: Proceedings of the 1st Conference on Language Modeling (COLM), 2024.
- [10] C. S. Xia, Y. Deng, S. Dunn, L. Zhang, Demystifying LLM-based software engineering agents, *Proceedings of the ACM on Software Engineering* 2 (FSE) (2025) 801–824. doi:10.1145/3715754.
- [11] M. Cemri, M. Z. Pan, S. Yang, L. A. Agrawal, B. Chopra, R. Tiwari, K. Keutzer, A. Parameswaran, D. Klein, K. Ramchandran, M. Zaharia, J. E. Gonzalez, I. Stoica, Why do multi-agent LLM systems fail?, in: Proceedings of the 39th Conference on Neural Information Processing Systems Datasets and Benchmarks Track (NeurIPS), 2025.
- [12] Y. Zhang, H. Ruan, Z. Fan, A. Roychoudhury, Autocoderover: Autonomous program improvement, in: Proceedings of the 33rd ACM SIGSOFT International Symposium on Software Testing and Analysis (ISSTA), 2024, pp. 1592–1604.
- [13] C. E. Jimenez, J. Yang, A. Wettig, S. Yao, K. Pei, O. Press, K. R. Narasimhan, SWE-bench: Can language models resolve real-world GitHub issues?, in: Proceedings of the 12th International Conference on Learning Representations (ICLR), 2024.
- [14] M. Chen, J. Tworek, H. Jun, Q. Yuan, H. P. D. O. Pinto, J. Kaplan, H. Edwards, Y. Burda, N. Joseph, G. Brockman, et al., Evaluating large language models trained on code, *arXiv preprint arXiv:2107.03374* (2021).
- [15] S. Chen, Y. He, S. Jana, B. Ray, Red teaming program repair agents: When correct patches can hide vulnerabilities, *arXiv preprint arXiv:2509.25894* (2025).
- [16] W. Wang, W. Ma, Q. Hu, Y. Zhang, J. Sun, B. Wu, Y. Liu, G. Xu, L. Jiang, VulnRepairEval: An exploit-based evaluation framework for assessing large language model vulnerability repair capabilities, *arXiv preprint arXiv:2509.03331* (2025).
- [17] S. Zhang, M. Yin, J. Zhang, J. Liu, Z. Han, J. Zhang, B. Li, C. Wang, H. Wang, Y. Chen, Q. Wu, Which agent causes task failures and when? on automated failure attribution of LLM multi-agent systems, in: Proceedings of the 42nd International Conference on Machine Learning (ICML), 2025.
- [18] G. Zhang, J. Wang, J. Chen, W. Zhou, K. Wang, S. Yan, AgenTracer: Who is inducing failure in the LLM agentic systems?, in: Proceedings of the 14th International Conference on Learning Representations (ICLR), 2026.
- [19] E. Kuzmina, S. Pervez, S. E. Hosseini, M. Shahbaz, A. Akhunzada, Spring framework benchmarking utility for static application security testing (SAST) tools, *IEEE Internet of Things Journal* 12 (22) (2025) 46863–46877.
- [20] N. Tihanyi, T. Bisztray, M. A. Ferrag, B. Cherif, R. A. Dubniczky, R. Jain, L. C. Cordeiro, Vulnerability detection: From formal verification to large language models and hybrid approaches: A comprehensive overview, in: *Adversarial Example Detection and Mitigation Using Machine Learning*, Springer, 2026, pp. 33–47. doi:10.1007/978-3-031-99447-0_3.
- [21] W. Bai, W. Muhammad, Z. Rasheed, J. Peltonen, P. Abrahamsson, Dataset for the Paper: When Passing Tests Hides Vulnerabilities: An Empirical Study of Silent Failures in Agentic Systems (2026). doi:10.5281/zenodo.20831595.
- [22] M. L. Siddiq, J. C. Santos, SecurityEval dataset: mining vulnerability examples to evaluate machine learning-based code generation techniques, in: Proceedings of the 1st International Workshop on Mining Software Repositories Applications for Privacy and Security (MSR4P&S), 2022, pp. 29–33.
- [23] G. Bhandari, A. Naseer, L. Moonen, CVEfixes: automated collection of vulnerabilities and their fixes from open-source software, in: Proceedings of the 17th International Conference on Predictive Models and Data Analytics in Software Engineering (PROMISE), 2021, pp. 30–39.
- [24] D. Huang, J. M. Zhang, M. Luck, Q. Bu, Y. Qing, H. Cui, Agentcoder: Multi-agent-based code generation with iterative testing and optimisation, *arXiv preprint arXiv:2312.13010* (2023).
- [25] P. Gauthier, Aider: AI pair programming in your terminal, <https://aider.chat>, accessed: 2026-06-17 (2024).
- [26] X. Wang, B. Li, Y. Song, F. F. Xu, X. Tang, M. Zhuge, J. Pan, Y. Song, B. Li, J. Singh, H. H. Tran, F. Li, R. Ma, M. Zheng, B. Qian, Y. Shao, N. Muennighoff, Y. Zhang, B. Hui, J. Lin, R. Brennan, H. Peng, H. Ji, G. Neubig, OpenHands: An open platform for AI software developers as generalist agents, in: Proceedings of the 13th International Conference on Learning Representations (ICLR), 2025.
- [27] G. Dolcetti, V. Arceri, E. Iotti, S. Maffei, A. Cortesi, E. Zaffanella, Helping LLMs improve code generation using feedback from testing and static analysis, *Discover Artificial Intelligence* 6 (1) (2026) 314.
- [28] K. M. MacQueen, E. McLellan, K. Kay, B. Milstein, Cookbook development for team-based qualitative analysis, *Cam Journal* 10 (2)

- (1998) 31–36.
- [29] M. Q. Patton, Qualitative research & evaluation methods: Integrating theory and practice, Sage publications, 2014.
 - [30] J. Corbin, A. Strauss, Basics of Qualitative Research: Techniques and Procedures for Developing Grounded Theory, 4th Edition, Sage Publications, 2014.
 - [31] K.-J. Stol, P. Ralph, B. Fitzgerald, Grounded theory in software engineering research: a critical review and guidelines, in: Proceedings of the 38th International Conference on Software Engineering (ICSE), 2016, pp. 120–131.
 - [32] B. Glaser, A. Strauss, Discovery of grounded theory: Strategies for qualitative research, Routledge, 2017.
 - [33] J. Cohen, A coefficient of agreement for nominal scales, Educational and Psychological Measurement 20 (1) (1960) 37–46.
 - [34] S. Han, Q. Zhang, Y. Yao, W. Jin, Z. Xu, LLM multi-agent systems: Challenges and open problems, arXiv preprint arXiv:2402.03578 (2024).
 - [35] G. Bansal, J. W. Vaughan, S. Amershi, E. Horvitz, A. Fourney, H. Mozannar, V. Dibia, D. S. Weld, Challenges in human-agent communication, arXiv preprint arXiv:2412.10380 (2024).
 - [36] H. Pearce, B. Tan, B. Ahmad, R. Karri, B. Dolan-Gavitt, Examining zero-shot vulnerability repair with large language models, in: 2023 IEEE Symposium on Security and Privacy (SP), IEEE, 2023, pp. 2339–2356.
 - [37] G. Dolcetti, E. Iotti, A dual perspective review on large language models and code verification, Frontiers in Computer Science 7 (2025) 1655469.
 - [38] M. Puvvadi, S. K. Arava, A. Santoria, S. S. P. Chennupati, H. V. Puvvadi, Coding agents: A comprehensive survey of automated bug fixing systems and benchmarks, in: 2025 IEEE 14th International Conference on Communication Systems and Network Technologies (CSNT), IEEE, 2025, pp. 680–686.
 - [39] P. Bisconti, M. Galisai, F. Pierucci, M. Bracale, M. Prandi, Beyond single-agent safety: A taxonomy of risks in LLM-to-LLM interactions, arXiv preprint arXiv:2512.02682 (2025).
 - [40] S. Kaniewski, F. Schmidt, M. Enzweiler, M. Menth, T. Heer, A systematic literature review on detecting software vulnerabilities with large language models, ACM Trans. Softw. Eng. Methodol. (May 2026). doi:10.1145/3815425.
 - [41] C. Wohlin, P. Runeson, M. Höst, M. C. Ohlsson, B. Regnell, A. Wesslén, et al., Experimentation in software engineering, Vol. 236, Springer, 2012.
 - [42] J.-L. Peng, S. Cheng, E. Diau, Y.-Y. Shih, P.-H. Chen, Y.-T. Lin, Y.-N. Chen, A survey of useful LLM evaluation, arXiv preprint arXiv:2406.00936 (2024).
 - [43] W. Wang, D. Zhang, T. Feng, B. Wang, J. Tang, Battleagentbench: A benchmark for evaluating cooperation and competition capabilities of language models in multi-agent systems, arXiv preprint arXiv:2408.15971 (2024).
 - [44] L. Hammond, A. Chan, J. Clifton, J. Hoelscher-Obermaier, A. Khan, E. McLean, C. Smith, W. Barfuss, J. Foerster, T. Gavenčiak, et al., Multi-agent risks from advanced AI, arXiv preprint arXiv:2502.14143 (2025).
 - [45] T. Masterman, S. Besen, M. Sawtell, A. Chao, The landscape of emerging AI agent architectures for reasoning, planning, and tool calling: A survey, arXiv preprint arXiv:2404.11584 (2024).
 - [46] A. Reid, S. O’Callaghan, L. Carroll, T. Caetano, Risk analysis techniques for governed LLM-based multi-agent systems, arXiv preprint arXiv:2508.05687 (2025).
 - [47] J. Becker, Multi-agent large language models for conversational task-solving, arXiv preprint arXiv:2410.22932 (2024).
 - [48] J. tse Huang, J. Zhou, T. Jin, X. Zhou, Z. Chen, W. Wang, Y. Yuan, M. Lyu, M. Sap, On the resilience of LLM-based multi-agent collaboration with faulty agents, in: Proceedings of the 42nd International Conference on Machine Learning (ICML), 2025.
 - [49] R. Widayarsi, M. Weyssow, I. C. Irsan, H. W. Ang, F. Liauw, E. L. Ouh, L. K. Shar, H. J. Kang, D. Lo, Let the trial begin: A mock-court approach to vulnerability detection using LLM-based agents, in: Proceedings of the IEEE/ACM 48th International Conference on Software Engineering (ICSE), 2026.
 - [50] Y. Wu, N. Jiang, H. V. Pham, T. Lutellier, J. Davis, L. Tan, P. Babkin, S. Shah, How effective are neural networks for fixing security vulnerabilities, in: Proceedings of the 32nd ACM SIGSOFT International Symposium on Software Testing and Analysis (ISSTA), 2023, pp. 1282–1294.
 - [51] A. Mahmud, Y. Rawajfih, F. Wu, A systematic evaluation of AI-generated security patches: Analyzing and exploiting AI-generated fixes, in: 2025 IEEE International Conference on Artificial Intelligence Testing (AITest), IEEE, 2025, pp. 224–231.
 - [52] E. Low, C. Cheh, B. Chen, Repairing infrastructure-as-code using large language models, in: 2024 IEEE Secure Development Conference (SecDev), IEEE, 2024, pp. 20–27.
 - [53] R. Meliala, C. Lim, J. Andreas, Integrating security testing in CI/CD pipelines: Current trends from literature and market, in: Proceedings of the 9th International Conference on Informatics and Computing (ICIC), IEEE, 2024, pp. 1–6.